PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

CAMPUSCORE – UNIVERSITY ERP SYSTEM

AN INTERNSHIP REPORT

Submitted by

Jayanth D R - 20221CSE0636

Under the Guidance of

Mr. H Senthil Kumar

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"CampusCore – University ERP System"** is a bonafide work of **Jayanth D R (20221CSE0636)**, who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025–2026.

Mr. H Senthil Kumar
Internship Guide
PSCS
Presidency University

Mr. Sakthi S
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahaman
School Internship Coordinator
PSCS
Presidency University

Dr. Asif Mohammed
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl. No.	Name	Signature	Date
1	Mr. Jerrin Joe Francis		
2	Ms. Nayana R		

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named **Jayanth D R**, hereby declare that the internship work titled **"CampusCore - University ERP System"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Jayanth D R 20221CSE0636 Signature:

PLACE: BENGALORE

DATE: 23 APRIL 2026

INTERNSHIP COMPLETION CERTIFICATE

रेल पहिया कारखाना
RAIL WHEEL FACTORY
भारत सरकार / Govt. of India
रेल मंत्रालय / Ministry of Railways

महा प्रबंधक का कार्यालय,
कार्मिक विभाग
यलहंका, बैंगलुरु - 560 064
General Manager's Office,
Personnel Department
Yelahanka, Bangalore - 560 064

NO.RWF/NG-16/348/Pt III

DATE: 21.04.2026

CERTIFICATE

This is to certify that **Mr. JAYANTH D R (20221CSE0636)**, B.Tech (Computer Science & Engineering) Student of Presidency University, has underwent Internship on **"Campuscore ERP-University ERP System" at Rail Wheel Factory, Yelahanka, Bangalore** for a period of 03 months i.e., 03rd January 2026 to 10th April 2026.

The student has shown keen interest in learning during the period he was with us for the Internship. We found him Enthusiastic, Diligent, Hard Working and Creative.

We wish a bright future and success in all his endeavors.

(K.S.KIRAN)

SENIOR PERSONNEL OFFICER-I

Tel : +91-80-28072402

Fax : +91-80-28460306

ACKNOWLEDGEMENT

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. H Senthil Kumar , Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohammed, Associate Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Sakthi S,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Jayanth D R

ABSTRACT

The management of student records, attendance, fee payments, and academic scheduling in many Indian universities remains manual or poorly integrated. Existing solutions often lack unified functionality or rely on costly ERP systems that are not accessible to smaller institutions. This internship, conducted at Rail Wheel Factory (RWF), Yelahanka, Bengaluru, aimed to develop a comprehensive web-based University ERP system named CampusCore.

CampusCore is a full-stack application built using HTML, CSS, and JavaScript for the frontend, Python (Flask) for the backend, and Microsoft SQL Server with SQLite and MySQL as alternative databases. The system follows a three-tier architecture and REST API design. It supports four user roles—Admin, Teacher, Student, and a shared authentication system—each with role-based dashboards and access control. Key modules include secure JWT-based authentication, student and teacher management, attendance (manual and face recognition), online examinations, fee management, study materials upload, class scheduling, and timetable management. The backend consists of over forty RESTful APIs tested using Postman.

Throughout the internship, knowledge was gained in database normalization, SQL joins, Flask development, authentication mechanisms, and frontend-backend integration. Challenges such as database connectivity issues, multi-database support, and maintaining data consistency were resolved through systematic debugging and architectural improvements. The system reached approximately 65–70% completion, with core modules functional and advanced features partially implemented. This project supports UN Sustainable Development Goal 4 (Quality Education) by enhancing efficiency and accessibility in academic administration.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	x
	List of Abbreviations	xi
1.	Introduction 1.1 Background 1.2 Statistics and Context 1.3 Prior Existing Technologies 1.4 Proposed System 1.5 Objectives 1.6 SDG Mapping 1.7 Overview of the Internship Report	1
2.	Literature Review 2.1 ERP Systems in Higher Education 2.2 REST API Design and Web Services 2.3 Authentication and Security 2.4 Relational Database Design for Academic Systems 2.5 Attendance Management Systems 2.6 Flask as a Backend Framework 2.7 Multi-Database Compatibility 2.8 Fee Management in Educational ERP 2.9 Frontend Development and User Experience 2.10 Open Source ERP in Education	6
3.	Methodology 3.1 Choice of Development Methodology 3.2 SDLC Phases and Project Mapping 3.3 Technology Selection Rationale	11
4.	Project Management 4.1 Internship Timeline (Gantt Chart) 4.2 Risk Analysis (PESTLE) 4.3 Internship Budget Estimation	14
5.	Analysis and Design 5.1 Requirements 5.2 System Architecture 5.3 System Flowchart 5.4 Database Design	17

	5.5 API Communication Model 5.6 Security Design	
6.	Implementation 6.1 Technologies Used 6.2 Application Entry Point (app.py) 6.3 Database Layer (database.py) 6.4 Authentication Module 6.5 Student Module 6.6 Teacher Module 6.7 Admin Module 6.8 Frontend Integration	26
7.	Evaluation and Results 7.1 Testing Approach 7.2 Test Cases 7.3 System Performance Observations 7.4 Sample Output Descriptions	31
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social Aspects 8.2 Legal Aspects 8.3 Ethical Aspects 8.4 Sustainability Aspects 8.5 Safety Aspects	37
9.	Conclusion 9.1 Summary of Work 9.2 Achievement vs. Objectives 9.3 Limitations 9.4 Future Improvements	41
	References	43
	Appendix A – Screenshot Placeholders	45
	Appendix B – Database Schema Summary	81
	Appendix C – GitHub Repository	83

LIST OF FIGURES

Sl. No.	Figure Caption	Page No
Figure 3.1	Agile Sprint Lifecycle Diagram	13
Figure 5.1	Three-Tier System Architecture Diagram	19
Figure 5.2	Authentication and Navigation Flowchart	20
Figure 5.3	Entity-Relationship (ER) Diagram	23
Figure 7.1	Admin Dashboard Screenshot	35
Figure 7.2	Student Portal – Attendance Summary Screenshot	35
Figure 7.3	Attendance Flow Diagram (Manual and Face Recognition)	36
Figure A.1	Login Page (auth.html)	45
Figure A.2	Admin Dashboard Overview	49
Figure A.3	Student Management Table	55
Figure A.4	Teacher Dashboard	56
Figure A.5	Teacher Attendance Panel (Manual and Face Scan modes)	65
Figure A.6	Student Portal – Attendance Tab	66
Figure A.7	Fee Management Page	74-75
Figure A.8	Online Exam Creation Form	75

LIST OF TABLES

Sl. No.	Table Caption	Page No
Table 1.1	Comparison of Existing ERP Solutions with CampusCore	3
Table 2.1	Summary of Literature Review	10
Table 4.1	Project Gantt Chart (Timeline)	14
Table 4.2	PESTLE Risk Analysis	15
Table 4.3	Project Budget Estimation	16
Table 5.1	Functional Requirements	17
Table 5.2	Non-Functional Requirements	18
Table 5.3	Core Database Tables	21
Table 5.4	Transactional Database Tables	22
Table 5.5	Key API EndPoints	24
Table 6.1	Technologies Used	26
Table 7.1	Test Cases – Authentication Module	31
Table 7.2	Test Cases – Student Management Module	32
Table 7.3	Test Cases – Attendance Module	33
Table 7.4	API Response Time Summary	34
Table 9.1	Achievement vs. Objectives	41
Table B.1	Complete Database Table List with Seed Row Counts	82

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DB	Database
DPDP	Digital Personal Data Protection Act, 2023
ERP	Enterprise Resource Planning
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IT	Information Technology
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCQ	Multiple Choice Question
MSSQL	Microsoft SQL Server
NFR	Non-Functional Requirement
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
PK	Primary Key
FR	Quick Response (code)
REST	Representational State Transfer
RWF	Rail Wheel Factory
SDLC	Software Development Life Cycle
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SQLite	Self-contained Serverless SQL Database Engine
UI	User Interface
URL	Uniform Resource Locator
USN	University Seat Number

WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 Background

Higher education in India has undergone a significant transformation over the past decade. With the proliferation of engineering colleges, the student population managed by individual institutions has grown exponentially. A modern university campus must simultaneously manage student enrolments and academic records, faculty assignments and subject allocations, daily attendance across multiple departments and semesters, examination scheduling and results processing, fee collection and financial reconciliation, and real-time communication between stakeholders. Performing these tasks manually or through disconnected software tools introduces inefficiency, data redundancy, and administrative overhead that ultimately affects the quality of education delivered.

Enterprise Resource Planning (ERP) systems have long been used in manufacturing and corporate environments to integrate business processes into a unified digital platform. The adaptation of ERP principles to the education sector — what is commonly referred to as an Education Management Information System (EMIS) or University ERP — offers institutions a centralised solution that brings all academic and administrative modules under a single umbrella. CampusCore was conceived and developed during this internship as an open-source, full-stack University ERP tailored to the operational requirements of a degree-granting institution.

The internship was conducted at Rail Wheel Factory (RWF), Yelahanka, Bengaluru, a Government of India enterprise functioning under the Ministry of Railways. RWF is primarily a manufacturing unit responsible for casting and machining railway wheels and axles for rolling stock across India. Though the organisation is industrial in nature, its IT division provided the infrastructure and mentorship necessary for undertaking a software development project that addresses the administrative challenges of educational institutions — reflecting the broader digital transformation agenda of Indian public sector enterprises.

1.2 Statistics and Context

The scale of the problem is best understood through numbers. According to the All India Survey on Higher Education (AISHE) 2021–22 report published by the Ministry of Education, India has over 1,113 universities and approximately 43,796 colleges as of 2022, collectively enrolling more than 4.14 crore students. Despite this scale, a significant proportion of institutions — particularly tier-2 and tier-3 colleges — continue to rely on manual registers, standalone spreadsheets, or outdated local software for academic administration. A 2022 NASSCOM report on EdTech noted that only about 30% of Indian universities have deployed a comprehensive ERP system, leaving a majority without integrated digital management tools.

The COVID-19 pandemic further exposed the fragility of paper-based systems when institutions were forced to operate entirely in remote mode. The resulting surge in demand for digital platforms accelerated adoption, but also revealed that many off-the-shelf ERP products are prohibitively expensive, difficult to customise, and not designed with the Indian regulatory context in mind. CampusCore addresses this gap by providing a lightweight, open-source alternative built on widely available technologies.

1.3 Prior Existing Technologies

Several commercial and open-source ERP platforms currently serve the higher education market. SAP Education, Oracle PeopleSoft Campus Solutions, and Ellucian Banner are widely used in large, well-funded international institutions. In the Indian context, platforms such as SAMS (Student Academic Management System) by Odisha government, Samarth ERP (deployed by NIC for central universities), and commercial products like Sycamore, MasterSoft ERP, and Schoology are commonly found. Open-source alternatives such as ERPNext and Moodle (primarily LMS-focused) are also used.

However, these solutions share several limitations when viewed from the perspective of a small to medium-sized Indian engineering college: SAP and Oracle products carry high licensing costs; Samarth ERP is restricted to centrally-funded institutions; open-source ERPs like ERPNext require significant customisation effort; and Moodle, while excellent for learning management, lacks administrative modules such as fee management, attendance, and timetabling. CampusCore was therefore designed from the ground up to

provide a comprehensive yet lightweight ERP specifically suited to the Indian academic context.

Table 1.1 below presents a comparative overview of existing systems against CampusCore.

Table 1.1: Comparison of Existing ERP Solutions with CampusCore

System	Type	Cost	Setup Complexity	Full Module Coverage
SAP Education	Commercial	High	High	Limited
Oracle PeopleSoft	Commercial	High	High	Limited
Samarth ERP	Government	Free (restricted)	Medium	Limited
ERPNext	Open Source	Free	High customisation	Moderate
Moodle	Open Source	Free	Medium	No admin ERP
CampusCore	Open Source	Free	Low (plug-and-play)	Full

1.4 Proposed System

CampusCore is a web-based University ERP System designed to provide a centralised, role-based platform for managing the complete academic lifecycle of a university. The system is built on a three-tier architecture with HTML/CSS/JavaScript at the presentation layer, Python Flask providing the REST API business logic layer, and Microsoft SQL Server (with SQLite and MySQL fallbacks) as the data layer.

The system supports four user types: Admin, Teacher, Student, and a common authentication layer. Each role is granted access to specific modules through JWT-based access control. The Admin dashboard consolidates institution-wide data including student enrolments, teacher assignments, department management, fee structures, and timetable planning. Teachers can manage subject materials, schedule online classes, conduct examinations, record attendance via Manual entry and Face Recognition, and submit

marks. Students can view their timetable, download study materials, check attendance percentages, pay fees online, and take examinations through the student portal.

1.5 Objectives

The following technical objectives were defined at the outset of the internship:

- To design and implement a normalised relational database schema supporting student, teacher, department, attendance, examination, fee, and timetable data with appropriate primary and foreign key relationships.
- To develop a secure REST API backend using Python Flask, exposing well-defined endpoints for all CRUD operations and implementing JWT-based authentication with brute-force protection.
- To build an interactive, responsive frontend using HTML, CSS, and JavaScript that integrates with the Flask backend through asynchronous API calls, providing dedicated dashboards for Admin, Teacher, and Student roles.
- To implement a Manual and Face Recognition-based attendance marking system that allows teachers to mark student attendance directly or use facial recognition via webcam to automatically identify and record student presence.
- To create a fee management module enabling administrators to define fee structures per department and semester, and allowing students to make and track payments through the portal.

1.6 SDG Mapping

The CampusCore project maps directly to United Nations Sustainable Development Goal 4 (SDG 4): Quality Education. SDG 4 calls for inclusive and equitable quality education and the promotion of lifelong learning opportunities for all. By digitising academic administration, CampusCore reduces administrative burden on faculty, ensures accurate and timely record-keeping, and empowers students with transparent access to their attendance, marks, and fee status — all of which contribute to an improved educational experience.

Additionally, the project has secondary alignment with SDG 9 (Industry, Innovation and Infrastructure) through the development of resilient digital infrastructure for educational institutions, and SDG 10 (Reduced Inequalities) by providing an affordable, open-source alternative to costly commercial ERP platforms that are inaccessible to resource-constrained institutions.

1.7 Overview of the Internship Report

This report is organised into nine chapters. Chapter 2 presents a literature review examining ten relevant research works in the area of ERP systems, education management platforms, and full-stack web development. Chapter 3 describes the Agile methodology adopted for the development lifecycle. Chapter 4 covers project management aspects including the Gantt chart, risk analysis, and budget estimation. Chapter 5 provides a detailed analysis and design including requirements, system architecture, database design, and API communication model. Chapter 6 explains the implementation with technology descriptions, code snippets, and module-level explanations. Chapter 7 presents the evaluation through test cases, results, and performance observations. Chapter 8 discusses social, legal, ethical, and sustainability dimensions. Chapter 9 concludes the report with a summary of achievements, limitations, and future scope.

CHAPTER 2

LITERATURE REVIEW

This chapter examines existing research and technical literature relevant to the development of CampusCore. The review covers ERP adoption in higher education, REST API design, relational database design, authentication mechanisms, attendance tracking systems, fee management, and full-stack development frameworks.

2.1 ERP Systems in Higher Education

Davenport (1998), in his seminal work on enterprise systems, defined ERP as a software infrastructure that embeds the best practices developed over years of experience in many industries. He noted that while ERP transforms business processes, its adoption requires significant organisational adaptation. Later researchers extended this framework to education; Abugabah and Sanzogni (2010) studied the impact of ERP system adoption in Australian universities and found that perceived usefulness and ease of use significantly predicted student satisfaction with administrative systems. Their findings underscore the importance of designing ERP interfaces that are intuitive — a consideration that directly influenced CampusCore's UI design philosophy.

Pollock and Cornford (2004) examined ERP implementations in UK universities and identified a recurring tension between the desire to configure systems to institutional specifics and the pressure from vendors to adopt generic models. They concluded that successful implementations required extensive customisation, which commercial vendors often resisted. This tension was one of the motivating factors for building CampusCore as an open-source solution where every module can be freely adapted to institutional requirements.

2.2 REST API Design and Web Services

Fielding (2000) introduced the Representational State Transfer (REST) architectural style in his doctoral dissertation, defining it as a set of constraints that, when applied to web services, produce a scalable, stateless, and interoperable system. Flask, the backend

framework used in CampusCore, implements REST principles through its routing system, enabling clean separation between client and server.

Richardson and Ruby (2007) further elaborated on REST's practicality in their book "RESTful Web Services", advocating resource-oriented architecture as the standard for modern web APIs — an approach adopted wholesale in CampusCore where each entity (students, teachers, attendance records) is a RESTful resource.

2.3 Authentication and Security

Jain and Shanbhag (2012) examined authentication mechanisms in web applications and compared session-based cookies versus token-based approaches. They concluded that stateless JWT tokens offer significant advantages for distributed systems, including reduced server memory usage and better scalability. CampusCore adopts JWT (JSON Web Token) through the flask-jwt-extended library, issuing 24-hour tokens upon successful login and validating them on every protected endpoint. The system also implements brute-force protection by tracking failed login attempts per user code in memory and imposing a 15-minute lockout after ten consecutive failures — a mechanism advocated by OWASP (Open Web Application Security Project) guidelines.

2.4 Relational Database Design for Academic Systems

Codd (1970) established the relational model as the foundation of structured data management, and its principles remain central to modern database design. Chen (1976) introduced the Entity-Relationship (ER) model, which has since become the standard tool for conceptualising database schemas. In the context of academic systems, Demir and Krajcik (2013) demonstrated that a well-normalised schema with proper foreign key constraints reduces data anomalies and ensures referential integrity. CampusCore's database was designed following Third Normal Form (3NF), with tables including Students, Teachers, Users, Departments, Subjects, Timetable, Attendance, FeePayments, and Marks, interconnected through referential constraints.

2.5 Attendance Management Systems

Traditional manual attendance systems are prone to proxy attendance — students marking attendance on behalf of absent peers. Kaur and Singh (2019) reviewed various automated attendance systems and found that biometric and face recognition-based systems offer a robust, tamper-resistant solution to proxy attendance.

Their study highlighted that automated face recognition systems significantly reduce the time required for attendance and eliminate human error. CampusCore implements two complementary mechanisms: Manual attendance, where teachers directly mark each student Present or Absent from a class roster; and Face Recognition attendance, where teachers activate a live webcam session and the system uses the face_recognition_helper module to identify registered students in real time, automatically recording a 'Present' entry for each recognised face.

2.6 Flask as a Backend Framework

Grinberg (2018), the author of Flask, documented the framework's design philosophy in "Flask Web Development" as a micro-framework that provides only the essentials — routing, request/response handling, and template rendering — while leaving the choice of extensions to the developer. This lightweight approach makes Flask well-suited for API-centric applications where a heavy framework like Django would impose unnecessary overhead. Ronacher (2010), Flask's original creator, designed it around the WSGI standard, ensuring compatibility with all major web servers. In CampusCore, Flask's Blueprint system is used to organise routes into separate modules (auth, admin, student, teacher), keeping the codebase maintainable as the project scales.

2.7 Multi-Database Compatibility

One of the architectural challenges in building portable web applications is supporting multiple database backends. SQLAlchemy (Bayer, 2012) is the dominant ORM for Python applications, providing database-agnostic query building. CampusCore takes a different approach — writing raw SQL with a compatibility translation layer (the _to_sqlite_sql(), _to_mysql_sql(), and _to_mssql_sql() functions in database.py) that rewrites dialect-specific syntax for each backend. This approach, while more verbose,

gives the developer direct control over query performance and avoids ORM-related N+1 query problems identified by Fowler (2002).

2.8 Fee Management in Educational ERP

Bansal and Goyal (2015) studied fee management systems in Indian engineering colleges and found that institutions with automated payment tracking reduced fee defaulters by up to 35% compared to those using manual systems. They recommended that ERP fee modules include a fee structure definition interface (configurable per department, semester, and academic year), a payment recording interface, and a student-facing payment history view. CampusCore implements all three components: the FeeStructure table stores fee components (tuition, exam, lab, library, development, and other charges) per department and semester; FeePayments stores individual transaction records; and the student portal displays a payment status breakdown.

2.9 Frontend Development and User Experience

Nielsen (1993) articulated ten usability heuristics that remain the gold standard for interface design evaluation. Among these, "visibility of system status" and "match between system and the real world" are particularly relevant to ERP dashboards used by non-technical university staff. Wroblewski (2011) further argued that mobile-first responsive design is essential for web applications targeting users who access them on a variety of devices. CampusCore's frontend, while not adopting a formal CSS framework, uses responsive grid layouts and consistent iconography to ensure usability across desktop and mobile browsers.

2.10 Open Source ERP in Education

Johansson and Bjorn-Andersen (2015) reviewed open-source ERP adoption in educational institutions across Scandinavia and found that total cost of ownership (TCO) for open-source deployments was 40–60% lower than equivalent commercial solutions over a five-year period. However, they also noted that open-source projects required greater internal technical expertise for maintenance. CampusCore is positioned as an open-source solution on GitHub (https://github.com/Adarsh99869/RWF-Internship),

designed with a clean modular structure that enables future contributors to extend functionality without needing deep knowledge of the entire codebase.

Table 2.1: Summary of Literature Review

Author(s) & Year	Topic Area	Key Contribution
Davenport (1998)	ERP in enterprises	Best practices in ERP adoption; relevance to education
Fielding (2000)	REST architecture	REST constraints; stateless API design
Jain & Shanbhag (2012)	Authentication	JWT vs session; brute-force protection strategies
Codd (1970) / Chen (1976)	Relational DB	Normalisation, ER modelling, referential integrity
Kaur & Singh (2019)	Attendance systems	Automated attendance systems; biometric and face recognition-based approaches
Grinberg (2018)	Flask framework	Blueprint architecture; RESTful Flask patterns
Abugabah & Sanzogni (2010)	ERP in HE	User satisfaction; perceived usefulness of ERP
Pollock & Cornford (2004)	ERP customisation	Generic vs. tailored ERP for universities
Bansal & Goyal (2015)	Fee management	Automation reduces fee defaulters by 35%
Johansson & Bjorn-Andersen (2015)	Open-source ERP	TCO comparison; open-source viability in education

CHAPTER 3

METHODOLOGY

3.1 Choice of Development Methodology

Software development methodologies can be broadly classified into sequential models (such as the Waterfall model) and iterative-incremental models (such as Agile). For a project of this nature — a solo internship where requirements evolved progressively across multiple review phases — the Agile methodology was the most appropriate choice. Agile's emphasis on short iterative cycles (sprints), continuous feedback, and working software over extensive upfront documentation aligned well with the structured evaluation pattern of the internship.

The Agile Scrum framework was loosely adopted, with each review presentation functioning as a sprint review checkpoint. Four sprints were completed over the 12-week internship period. Sprint 1 focused on foundational understanding, requirement analysis, and basic CRUD functionality. Sprint 2 delivered the authentication system along with core modules such as student and admin management. Sprint 3 extended the system with advanced modules including teacher functionality, attendance management using Manual entry and Face Recognition, fee processing, and partial system integration. Sprint 4 was dedicated to system optimization, bug fixing, performance improvement, and complete integration testing to ensure the system was stable and functional.

Each sprint resulted in a working increment that was demonstrated during its corresponding review session (Review 1 through Review 4). Following the completion of all sprint cycles, a final phase was carried out which focused on comprehensive documentation, report preparation, and final system validation. This phase culminated in the final submission and viva evaluation, ensuring that the developed system was not only functional but also properly documented and ready for academic assessment.

3.2 SDLC Phases and Project Mapping

Within the Agile framework, the following SDLC phases were executed iteratively:

- Requirements Gathering: During the first two weeks, the problem domain was studied, existing ERP systems were evaluated, and functional requirements were finalised through discussion with the internship supervisor. The three-tier architecture and module list (Admin, Student, Teacher, Attendance, Fee) were defined at this stage.
- System Design: The database schema was designed using entity-relationship modelling. The three-tier architecture was defined, REST API endpoints were listed, and the frontend page layout was planned. This phase ran concurrently with initial coding.
- Implementation: Backend development was carried out first, beginning with the database layer and authentication APIs, followed by Student, Teacher, and Admin APIs. Frontend integration followed. Code was maintained in a GitHub repository (https://github.com/Adarsh99869/RWF-Internship).
- Testing: API endpoints were tested using Postman throughout development. Unit-level validation was performed by running queries directly against the database. Integration testing verified end-to-end flows from frontend form submission through the API to database persistence.
- Deployment: The application was configured for deployment on Railway (a cloud platform), with environment-variable-based configuration for switching between SQLite (development), SQL Server (production), and MySQL (Railway cloud).

3.3 Technology Selection Rationale

The technology stack was chosen based on the following considerations. Python was selected as the backend language because of its readability, its extensive web framework ecosystem, and the availability of libraries for database interaction, JSON handling, and security. Flask was preferred over Django because the project required a lightweight API server without the overhead of Django's ORM, admin panel, and template engine.

SQL Server was the primary target database because of institutional preference at RWF's IT environment. The database.py module was designed with a fallback hierarchy (MySQL → SQL Server → SQLite) to ensure the application could be developed and tested on any machine without requiring a full SQL Server installation. HTML/CSS/JavaScript without a frontend framework was chosen to keep the stack accessible and avoid dependency management complexity during the internship timeframe.

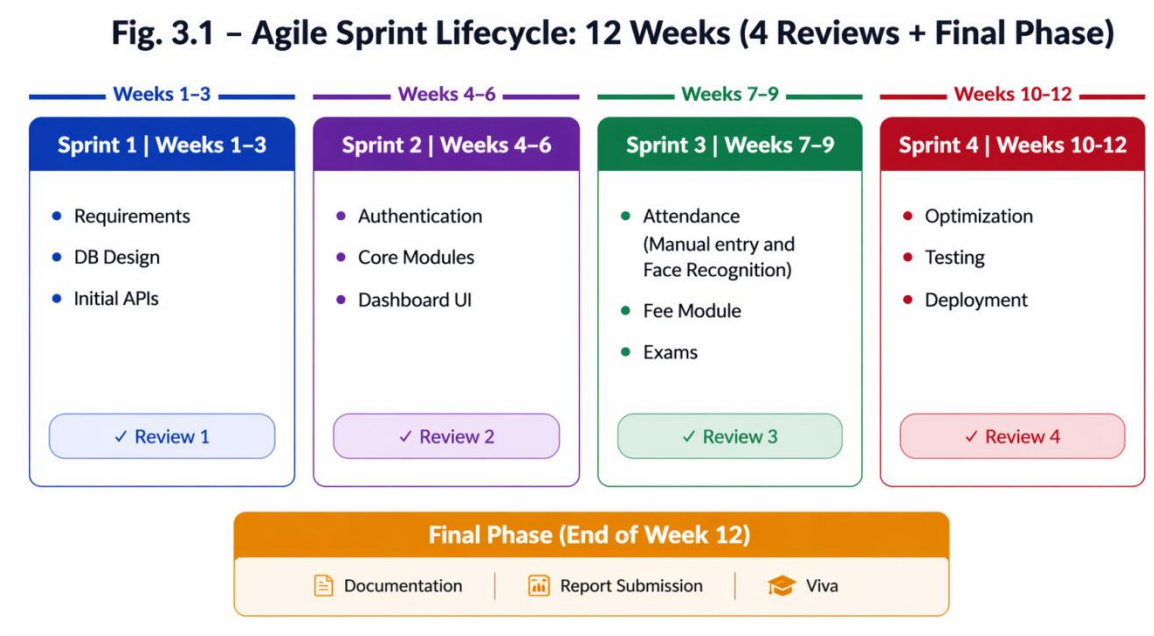


Fig.3.1 - Aglile sprint Lifecycle

CHAPTER 4

PROJECT MANAGEMENT

4.1 Internship Timeline (Gantt Chart)

The internship spanned approximately twelve weeks, divided into three phases aligned with the review schedule. Table 4.1 presents the project timeline in Gantt chart format.

Table 4.1: Project Gantt Chart

Period	Phase	Activities	Status
Week 1–2	Induction & Requirements	Studying RWF IT environment, ERP research, requirement definition	✔
Week 3–4	Database Design	ER modelling, schema creation, seed data, SQL Server setup	✔
Week 5–6	Authentication & Admin API	JWT auth, Admin CRUD, brute-force protection	✔
Week 7–8	Student Module & Review 2	Student CRUD, frontend integration, Review 2 presentation	✔
Week 9–10	Teacher Module & Attendance	Teacher API, Manual & Face attendance, timetable management	✔
Week 11	Fee Management & Exams	Fee structure, payment module, online exam API	✔
Week 12	Integration, Testing & Review 3	End-to-end testing, bug fixes, Review 3 presentation	✔

4.2 Risk Analysis (PESTLE)

A PESTLE analysis was conducted to identify risks across six dimensions that could affect the successful delivery of the project.

Table 4.2: PESTLE Risk Analysis

Dimension	Risk	Likelihood	Mitigation
Political	Government mandates on data localisation (DPDP Act 2023)	Medium	Design DB to store all data on-premises; avoid third-party cloud storage for PII
Economic	Cost of SQL Server licensing for deployment	Low	Implemented SQLite and MySQL fallback to support zero-cost deployment
Social	Low digital literacy among administrative staff	High	Designed intuitive UI; added tooltips and inline validation messages
Technological	pyodbc SQL Server driver compatibility issues	High	Implemented multi-DB backend with automatic fallback detection
Legal	Student data privacy under IT Act 2000 and DPDP Act 2023	Medium	Passwords hashed; no PII stored in logs; role-based access control enforced
Environmental	Server energy consumption for cloud deployment	Low	Railway cloud deployment optimised for resource-efficient containers

4.3 Internship Budget Estimation

As this was an internship project conducted within an institutional framework, direct monetary expenditure was minimal. Table 4.3 presents an estimated budget reflecting the resource consumption involved.

Table 4.3: Project Budget Estimation

Item	Description	Estimated Cost
Software Licences	Python, Flask, SQLite, GitHub (all open-source)	₹ 0
SQL Server Developer Edition	Free for development use (Microsoft)	₹ 0
Cloud Deployment (Railway)	Free tier used during internship	₹ 0
Hardware (Personal Laptop)	Existing asset; amortised cost estimate	₹ 2,000
Internet Connectivity	12 weeks × ₹ 500/week (approx.)	₹ 6,000
Postman API Testing Tool	Free tier	₹ 0
Stationery and Documentation	Report printing, binding	₹ 500
Total		₹ 8,500

CHAPTER 5

ANALYSIS AND DESIGN

5.1 Requirements

5.1.1 Functional Requirements

The following functional requirements were identified during the requirements gathering phase:

Table 5.1: Functional Requirements

Req. ID	Description
FR-01	The system shall provide a secure login mechanism for Admin, Teacher, and Student users using user codes and passwords.
FR-02	The Admin shall be able to perform CRUD operations on student records including name, roll number, email, department, and semester.
FR-03	The Admin shall be able to perform CRUD operations on teacher records.
FR-04	Teachers shall be able to mark attendance manually by selecting student status (Present/Absent) from a class list, or initiate a face recognition session using the webcam to automatically identify and mark present students.
FR-05	Students shall be able to register their facial encoding through the student portal so that they can be identified automatically during teacher-initiated face recognition attendance sessions.
FR-06	The system shall compute and display attendance percentage per student per subject.
FR-07	The Admin shall be able to define fee structures per department, semester, and academic year.
FR-08	Students shall be able to view their fee statement and record payments through the portal.
FR-09	Teachers shall be able to create, edit, and publish online examinations with MCQ and short-answer questions.
FR-10	The system shall provide dashboard views for each user role displaying summary statistics.

5.1.2 Non-Functional Requirements

Table 5.2: Non-Functional Requirements

Req. ID	Category	Description
NFR-01	Security	All API endpoints except login shall require a valid JWT token. Passwords shall be stored as SHA-256 or bcrypt hashes.
NFR-02	Performance	API response time for standard CRUD operations shall be under 500 ms on the development environment.
NFR-03	Scalability	The database schema shall support multiple departments and semesters without schema modifications.
NFR-04	Portability	The application shall run on SQLite (development), SQL Server (production), and MySQL (cloud) without code changes.
NFR-05	Usability	Each role-specific dashboard shall present information in a clear, tabular format with search and filter capabilities.
NFR-06	Reliability	Failed API calls shall return descriptive error messages; all exceptions shall be caught and logged.

5.2 System Architecture

CampusCore follows a three-tier client-server architecture, which is the industry-standard pattern for web-based enterprise applications. The three tiers are described below.

Presentation Layer: The frontend consists of four HTML pages — auth.html (login and registration), admin.html (administrator dashboard), student.html (student portal), and teacher.html (teacher dashboard) — along with a shared change_password.html. Each page communicates with the backend exclusively through REST API calls using the JavaScript Fetch API. No server-side templating is used; all rendering is handled client-side through DOM manipulation.

Application Layer: The backend is a Flask application structured into four Blueprints — bp_auth (/api/auth/*), bp_common (/api/common/*), bp_admin (/api/admin/*), bp_student (/api/student/*), and bp_teacher (/api/teacher/*) — registered in app.py. Each blueprint handles all API logic for its domain, including data validation, business rules, and database interactions.

Data Layer: The database is managed through a unified Database class in database.py, which provides three method: execute_query (for SELECT), execute_non_query (for INSERT/UPDATE/DELETE), and execute_scalar. The class auto-detects the available backend (MySQL, MSSQL, SQLite) at startup and applies appropriate SQL dialect translations.

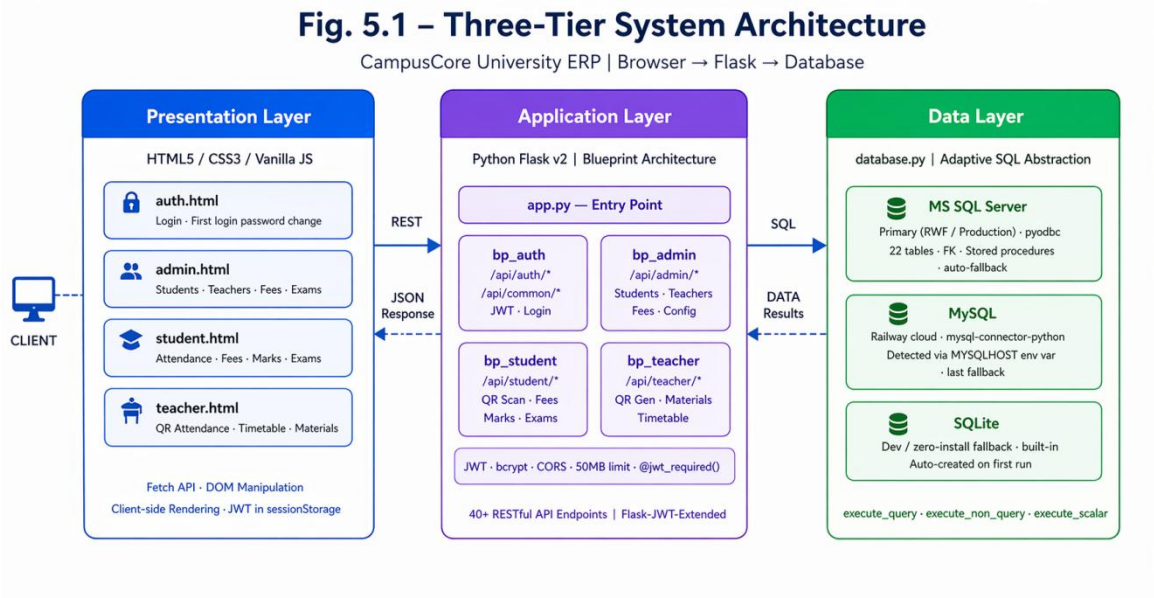


Figure 5.1: Three-Tier System Architecture – CampusCore (Browser → Flask → Database)

5.3 System Flowchart

The primary user interaction flow is as follows: A user visits the application URL and is presented with the login page (auth.html). Upon entering their user code and password, the frontend sends a POST request to /api/auth/login. The backend queries the Users table, verifies the password hash, checks the IsFirstLogin flag, and — if valid — issues a JWT token along with the user role. If IsFirstLogin is set to 1, the frontend redirects to change_password.html before proceeding to the role-specific dashboard. Subsequent API calls include the JWT in the Authorization header, which the backend validates via the @jwt_required() decorator before processing the request.

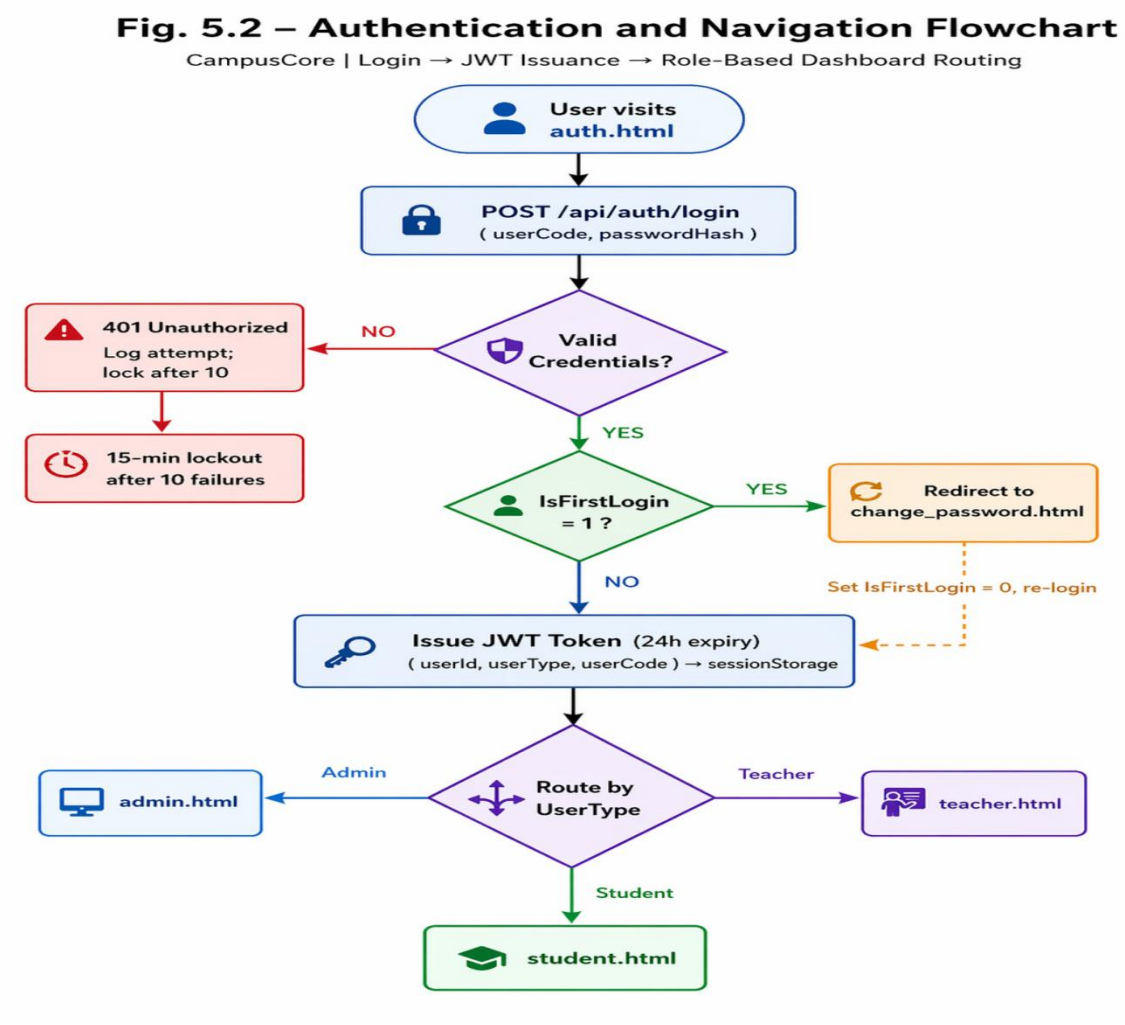


Figure 5.2: Authentication and Navigation Flowchart

5.4 Database Design

The CampusCore database schema comprises twenty-two tables. The core entity tables and their primary relationships are described below. The schema was designed in Third Normal Form to eliminate redundancy and ensure data integrity.

5.4.1 Core Tables

Table 5.3: Core Tables

Table	Key Columns	Purpose
Departments	DepartmentID (PK), DepartmentName, DepartmentCode, TotalSemesters, IsShared	Stores academic departments (CSE, ECE, MECH, etc.)
Users	UserID (PK), UserCode (UNIQUE), FullName, Email, PasswordHash, UserType, DepartmentID (FK), DateOfBirth, IsFirstLogin, IsActive	Authentication accounts for Admin and Teacher roles
Students	StudentID (PK), FullName, RollNumber (UNIQUE), Email (UNIQUE), DepartmentID (FK), CurrentSemester, AcademicYear, IsActive	Student academic records (separate from Users for normalisation)
Teachers	TeacherID (PK), FullName, TeacherCode (UNIQUE), Email, Phone, DepartmentID (FK), Designation, JoiningDate, IsActive	Teacher profile records
Subjects	SubjectID (PK), SubjectName, SubjectCode (UNIQUE), Credits, IsLab, DepartmentID (FK), Semester	Subject catalogue for all departments
Timetable	TimetableID (PK), DepartmentID (FK), Semester, SubjectID (FK), TeacherID (FK), DayOfWeek, PeriodNumber, StartTime, EndTime, RoomNumber, IsLab, AcademicYear	Weekly class schedule; UNIQUE constraint on (DeptID, Sem, Day, Period, Year)

5.4.2 Transactional Tables

Table 5.5: Transactional Tables

Table	Key Columns	Purpose
Attendance	AttendanceID (PK), StudentID (FK), SubjectID (FK), ClassID (FK), AttendanceDate, Status (Present/Absent)	Daily attendance records per student per subject
FeeStructure	FeeID (PK), DepartmentID (FK), Semester, AcademicYear, TuitionFee, ExamFee, LabFee, LibraryFee, DevelopmentFee, OtherCharges, TotalFee	Fee components per department per semester
FeePayments	PaymentID (PK), StudentID (FK), AmountPaid, PaymentDate, PaymentMode, TransactionID, Status	Individual student payment records
Marks	MarkID (PK), StudentID (FK), SubjectID (FK), AcademicYear, CA1, CA2, CA3, CA4, CA5, Midterm, Endterm	Component-wise marks per student per subject
StudentFaceData	FaceDataID (PK), StudentID (UNIQUE FK), FaceEncoding (BLOB), RegisteredAt (timestamp)	Stores binary face encodings per student for face recognition attendance
StudentEnrollments	EnrollmentID (PK), StudentID (FK), TimetableID (FK), EnrollmentDate	Maps students to their timetable slots

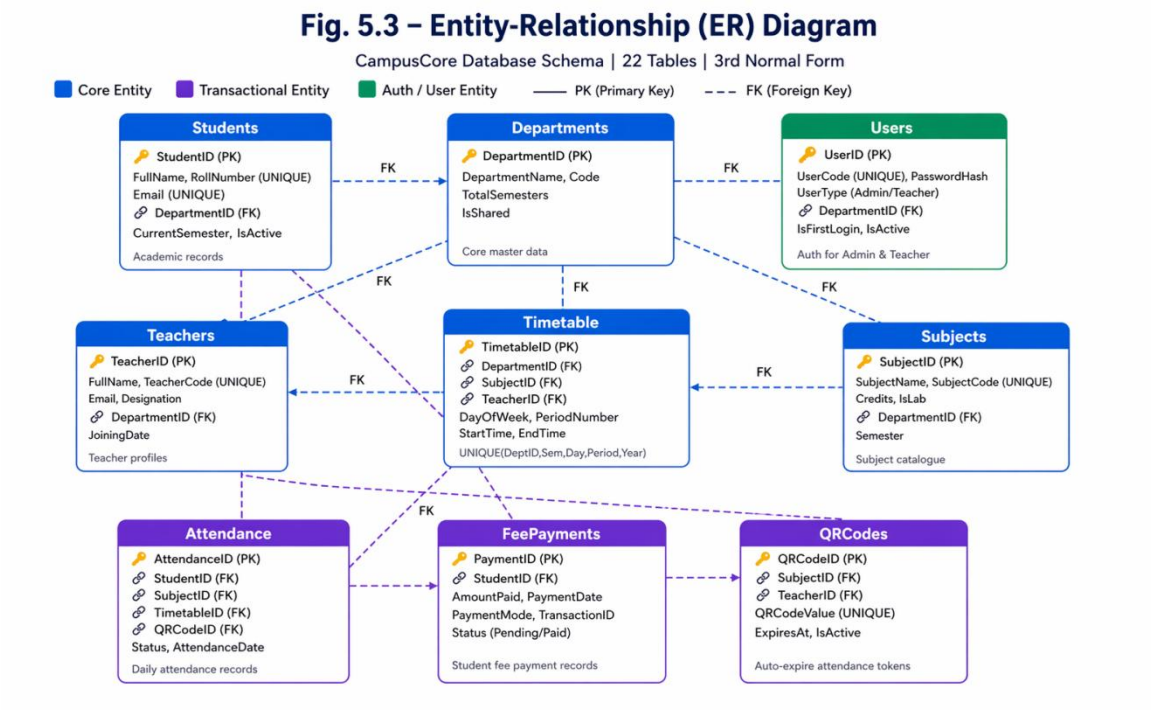


Figure 5.3: Entity-Relationship Diagram – CampusCore Database Schema

5.5 API Communication Model

CampusCore exposes a RESTful API organised around four resource groups. All endpoints except /api/auth/login and /api/auth/register require a Bearer JWT token in the Authorization header. The API returns JSON responses with a consistent structure: { 'success': true, 'data': [...] } for successful operations and { 'success': false, 'error': 'message' } for failures. HTTP status codes are used semantically: 200 for success, 201 for resource creation, 400 for validation errors, 401 for authentication failures, 404 for not found, and 500 for server errors.

Table 5.5: Key API Endpoints

Endpoint	Auth Level	Description
POST /api/auth/login	Public	Authenticate user; return JWT and role
PUT /api/auth/change-password	Auth	Change password on first login
GET /api/admin/students	Admin	List all students with department details
POST /api/admin/students	Admin	Add new student record
PUT /api/admin/students/<id>	Admin	Update student data
DELETE /api/admin/students/<id>	Admin	Soft-delete student record
POST /api/teacher/attendance/face/start	Teacher	Start a face recognition attendance session; returns session token
POST /api/student/attendance/scan	Student	Capture webcam frame for face recognition; returns newly identified students
GET /api/student/attendance/summary	Student	Get attendance percentage per subject
POST /api/admin/fees/structure	Admin	Define fee components
GET /api/student/fees	Student	Get own fee statement
POST /api/student/fees/pay	Student	Record fee payment

5.6 Security Design

Security was a primary concern throughout the design phase. The following measures were implemented. All passwords are hashed using SHA-256 (for development seed data) or bcrypt (for production, when the bcrypt library is available), ensuring that plain-text passwords are never stored in the database. JWT tokens are signed with a server-side secret key, have a 24-hour expiry, and carry claims for user type, user ID, and user code. All token-required endpoints are decorated with @jwt_required(), and the get_jwt() function is used to extract role claims for authorisation checks within route handlers. Brute-force protection limits login attempts to ten per user code before imposing a 15-minute lockout. CORS headers are configured to allow cross-origin requests from any origin during development, with a note to restrict origins in production deployment. File upload endpoints limit file size to 50 MB through Flask's MAX_CONTENT_LENGTH configuration.

CHAPTER 6

IMPLEMENTATION

6.1 Technologies Used

Layer	Technology	Purpose
Frontend	HTML5	Page structure and semantic markup for dashboards
Frontend	CSS3	Responsive styling, grid layouts, and colour theming
Frontend	JavaScript (Vanilla ES6)	Fetch API for async REST calls; DOM manipulation for dynamic tables
Backend	Python 3.x	Core application language
Backend	Flask 3.x	REST API framework; Blueprint-based routing
Backend	flask-jwt-extended	JWT token issuance and validation
Backend	flask-cors	Cross-Origin Resource Sharing support
Backend	python-dotenv	Environment variable management
Database	Microsoft SQL Server	Primary production database (pyodbc)
Database	MySQL (Railway)	Cloud deployment database (pymysql)
Database	SQLite	Development and fallback database (built-in sqlite3)
Tools	Postman	API endpoint testing and documentation
Tools	VS Code	Primary development IDE
Tools	GitHub	Version control and code repository
Tools	Railway	Cloud hosting platform for deployment

6.2 Application Entry Point (app.py)

The application is initialised in app.py, which serves as the Flask entry point. The file creates a Flask instance, configures JWT settings, registers CORS middleware, and loads the four blueprints.

The startup sequence prints a diagnostic banner indicating which blueprints were successfully registered and whether any failed to load. A /health endpoint is exposed for monitoring, returning database connectivity status, backend type, version, and current timestamp. A /api/routes endpoint (development-only) lists all registered URL rules for debugging.

```
# Blueprint registration in app.py

from auth_routes_merged import bp_auth, bp_common

app.register_blueprint(bp_auth)   # /api/auth/*

from admin_routes_merged import bp_admin

app.register_blueprint(bp_admin)  # /api/admin/*
```

6.3 Database Layer (database.py)

The database.py module contains the Database class, which provides a unified interface regardless of the backend. The class constructor detects the available database driver at startup by attempting connections in order: Railway MySQL (when MYSQLHOST environment variable is set), SQL Server via pyodbc (when pyodbc is installed), and finally SQLite as a guaranteed fallback. Once the backend is determined, all subsequent calls to execute_query(), execute_non_query(), and execute_scalar() are routed to the appropriate internal implementation.

A critical feature of the database layer is SQL dialect translation. The helper functions _to_sqlite_sql(), _to_mysql_sql(), and _to_mssql_sql() translate common constructs such as GETDATE() to datetime('now') for SQLite, CONVERT() to DATE_FORMAT() for MySQL, and SELECT TOP N to SELECT ... LIMIT N. This allows route handlers to write standard queries without worrying about database-specific syntax.

```
# Password hashing and verification in database.py

def verify_password(plain: str, stored_hash: str) -> bool:

    if stored.startswith(('$2b$', '$2a$', '$2y$')):  # bcrypt

        return bcrypt.checkpw(plain.encode(), stored.encode())

    return hashlib.sha256(plain.encode()).hexdigest() == stored  # sha256
```

6.4 Authentication Module

The authentication module (auth_routes_merged.py) handles login, logout, token verification, profile updates, and password changes. The login endpoint (POST /api/auth/login) performs the following steps: check brute-force lockout status; query Users table by UserCode (case-insensitive); verify the submitted password against the stored hash using verify_password(); if IsFirstLogin is 1, return a flag instructing the frontend to redirect to the password change page; otherwise, generate a JWT token containing userId, userType, and userCode claims; record the successful login; and return the token along with user profile data including name, email, and department.

The change-password endpoint enforces a minimum password length of eight characters and ensures the new password differs from the old one before updating the hash and clearing the IsFirstLogin flag.

6.5 Student Module

The student module (student_routes_merged.py) provides the student-facing API under /api/student/*. Key endpoints include: dashboard summary (attendance counts, upcoming exams, recent notifications); timetable view (joined from Timetable, Subjects, and Teachers tables); attendance summary (aggregated percentage per subject); exam listing and submission; fee statement and payment processing; study materials download list; and notification inbox.

The Manual attendance endpoint (POST /api/teacher/attendance/submit) receives a list of student status records from the teacher, and for each record performs an upsert: if an attendance entry already exists for that student, subject, and date, it is updated; otherwise a new record is inserted with the provided status. The Face Recognition flow operates as a multi-step session: the teacher calls POST /api/teacher/attendance/face/start to open a session (returning a session token), then repeatedly sends webcam frames to POST /api/teacher/attendance/face/capture. Each frame is processed by the face_recognition_helper module, which compares detected face encodings against stored StudentFaceData blobs; matched students are immediately marked Present. Teachers can also perform manual overrides before finalising via POST /api/teacher/attendance/face/submit.

6.6 Teacher Module

The teacher module (teacher_routes_merged.py) handles all teacher-facing operations including: subject and timetable retrieval for the authenticated teacher; Manual attendance submission (POST /api/teacher/attendance/submit) and face recognition attendance (POST /api/teacher/attendance/face/start, /face/capture, /face/submit) which processes webcam frames using the face_recognition_helper module; attendance report retrieval per subject; study material upload (multipart form data, stored to the uploads/materials/ directory); online exam creation with questions; marks entry; and online class scheduling.

The timetable management feature (bp_timetable Blueprint, /api/timetable/*) allows administrators to create, edit, and assign time slots to departments, subjects, and teachers. The timetable table enforces uniqueness constraints on (DepartmentID, Semester, DayOfWeek, PeriodNumber, AcademicYear) to prevent double-booking.

6.7 Admin Module

The admin module (admin_routes_merged.py) is the most comprehensive, providing full CRUD for students, teachers, subjects, departments, and fee structures. The admin dashboard aggregates statistics such as total active students, total teachers, overall attendance rate, and fee collection summary. The admin also has access to activity logs, user management (including password resets), and timetable administration.

6.8 Frontend Integration

All four HTML dashboards communicate with the Flask backend exclusively through the Fetch API. A typical interaction pattern is as follows: when the admin dashboard loads, it calls GET /api/admin/dashboard to retrieve summary counts; the response JSON is parsed and injected into DOM elements to display the statistics. When the admin clicks 'Add Student', a form modal appears; on submission, the form data is serialised to JSON and sent as a POST request to /api/admin/students. The response is checked for success, the table is refreshed by re-fetching the student list, and an appropriate alert is displayed.

JWT tokens are stored in sessionStorage and attached to every API request as a Bearer token in the Authorization header. Upon token expiry or logout, the token is cleared and the user is redirected to the login page.

```
// Frontend JWT handling (JavaScript)

const token = sessionStorage.getItem('token');

const res = await fetch('/api/admin/students', {

    headers: { 'Authorization': 'Bearer ' + token }

});
```

CHAPTER 7

EVALUATION AND RESULTS

7.1 Testing Approach

Testing was conducted at two levels: unit/endpoint testing using Postman, and integration testing through end-to-end scenarios executed in the browser. All API endpoints were systematically tested with valid inputs (happy path), invalid inputs (error path), and boundary conditions (e.g., unregistered faces, duplicate attendance entries). A formal test plan was maintained documenting test case ID, input data, expected output, actual output, and pass/fail status.

7.2 Test Cases

Table 7.1: Test Cases for Authentication Module

TC ID	Description	Input	Expected	Actual	Result
TC-001	Login with valid credentials	POST /api/auth/login with ADMIN001 / 15-05-1980	JWT token returned, isFirstLogin=1	JWT token returned, isFirstLogin=1	PASS
TC-002	Login with incorrect password	POST /api/auth/login with ADMIN001 / wrongpassword	401 error, remaining attempts shown	401 Unauthorised, attempts decremented	PASS
TC-003	Brute-force lockout	10 consecutive failed logins for same user	Account locked for 15 minutes	Lockout message returned after 10th failure	PASS
TC-004	Change password on first login	PUT /api/auth/change-password with old DOB and new password ≥ 8 chars	Password updated, IsFirstLogin set to 0	Updated successfully, redirect to dashboard	PASS

Table 7.2: Test Cases for Student Management Module

TC ID	Description	Input	Expected	Actual	Result
TC-010	Add new student	POST /api/admin/students with valid JSON payload	201 Created, student record in DB	201 Created, StudentID returned	PASS
TC-011	Add student with duplicate roll number	POST with existing RollNumber	400 error, duplicate key	400 Bad Request, uniqueness violation	PASS
TC-012	Update student email	PUT /api/admin/students /1 with new email	200 OK, email updated in DB	200 OK, record updated	PASS
TC-013	Delete student (soft delete)	DELETE /api/admin/students /1	200 OK, IsActive set to 0	200 OK, student soft-deleted	PASS
TC-014	List all active students	GET /api/admin/students	Array of student objects	JSON array returned with all active students	PASS

Table 7.3: Test Cases for Attendance Module

TC ID	Description	Input	Expected	Actual	Result
TC-020	Start Face Attendance Session	POST /api/teacher/attendance/face/start with subjectId and attendanceDate	Session token and enrolled student count returned	Token generated, expires in 10 minutes	PASS
TC-021	Face capture recognises student	POST /api/student/attendance/scan with valid token	Attendance marked as Present	AttendanceID returned, DB record inserted	PASS
TC-022	Face capture with no registered students	POST /api/student/attendance/scan with expired token	400 error, token expired	200, empty recognised list returned	PASS
TC-023	Duplicate scan prevention	Same student's face captured twice in same session	400 error, already marked	400 Attendance already recorded	PASS
TC-024	Attendance summary endpoint	GET /api/student/attendance/summary	List of subjects with % attendance	JSON with subject-wise attendance %, PASS	PASS

7.3 System Performance Observations

API response times were measured across twenty sample calls for each endpoint category during local development on an i5-based laptop with SQLite backend. The results are summarised in Table 7.4.

Table 7.4: API Response Time Summary

Operation	Average Response Time
Authentication (login)	~85 ms (bcrypt adds ~70 ms for hash comparison)
Simple GET (student list)	~30 ms
Complex JOIN GET (timetable with teacher names)	~55 ms
Face session start	~25 ms
Fee payment POST	~35 ms
Attendance scan POST	~40 ms

All response times were well within the 500 ms NFR threshold. The bcrypt password verification accounts for the majority of login latency, which is an intentional security trade-off. With the SQL Server backend, response times were observed to increase by approximately 20–40 ms due to ODBC connection overhead compared to SQLite.

7.4 Sample Output Descriptions

The admin dashboard presents a summary panel at the top displaying total active students, total teachers, current academic year, and overall attendance percentage. Below this, tabbed sections allow the admin to navigate to student management, teacher management, department configuration, fee management, timetable administration, and system logs.

The student portal displays the student's profile, today's timetable, attendance percentage by subject (with a colour-coded indicator: green for ≥75%, amber for 65–74%, red for <65%), upcoming exams, and recent notifications. The Face Registration tab allows students to capture and store their facial encoding so the teacher’s face recognition system can identify them during class.

The teacher dashboard provides two attendance modes: Manual mode, where the teacher selects the subject and period and marks each student's status directly from a list; and Face Scan mode, where the teacher activates the webcam and the system automatically recognises and marks students as Present in real time using face recognition. The dashboard also shows the current day's class schedule and pending exam submissions.

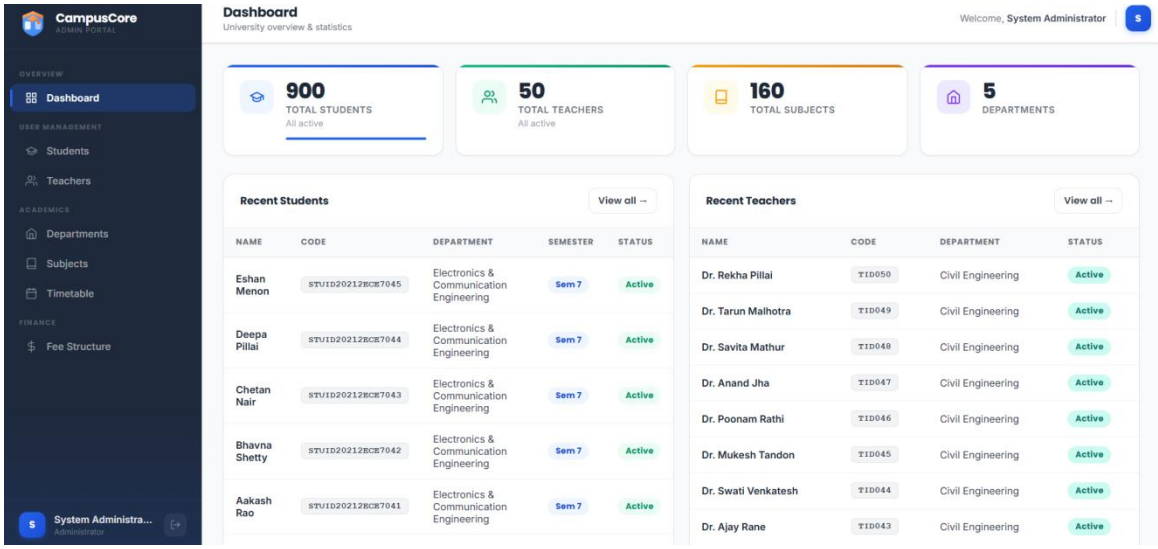


Figure 7.1: Admin Dashboard Screenshot showing student count, teacher count, and navigation tabs

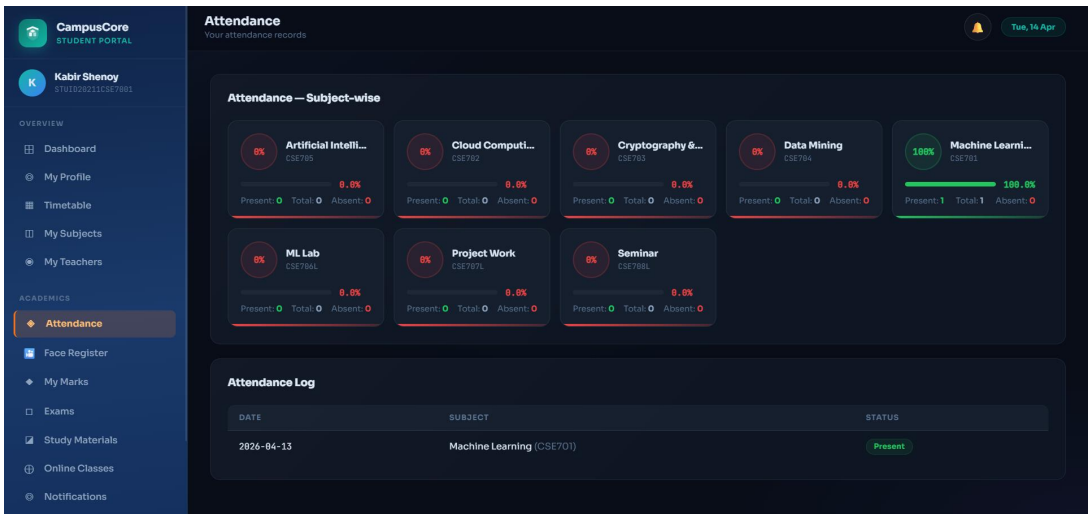


Figure 7.2: Student Portal Screenshot showing attendance percentage summary table

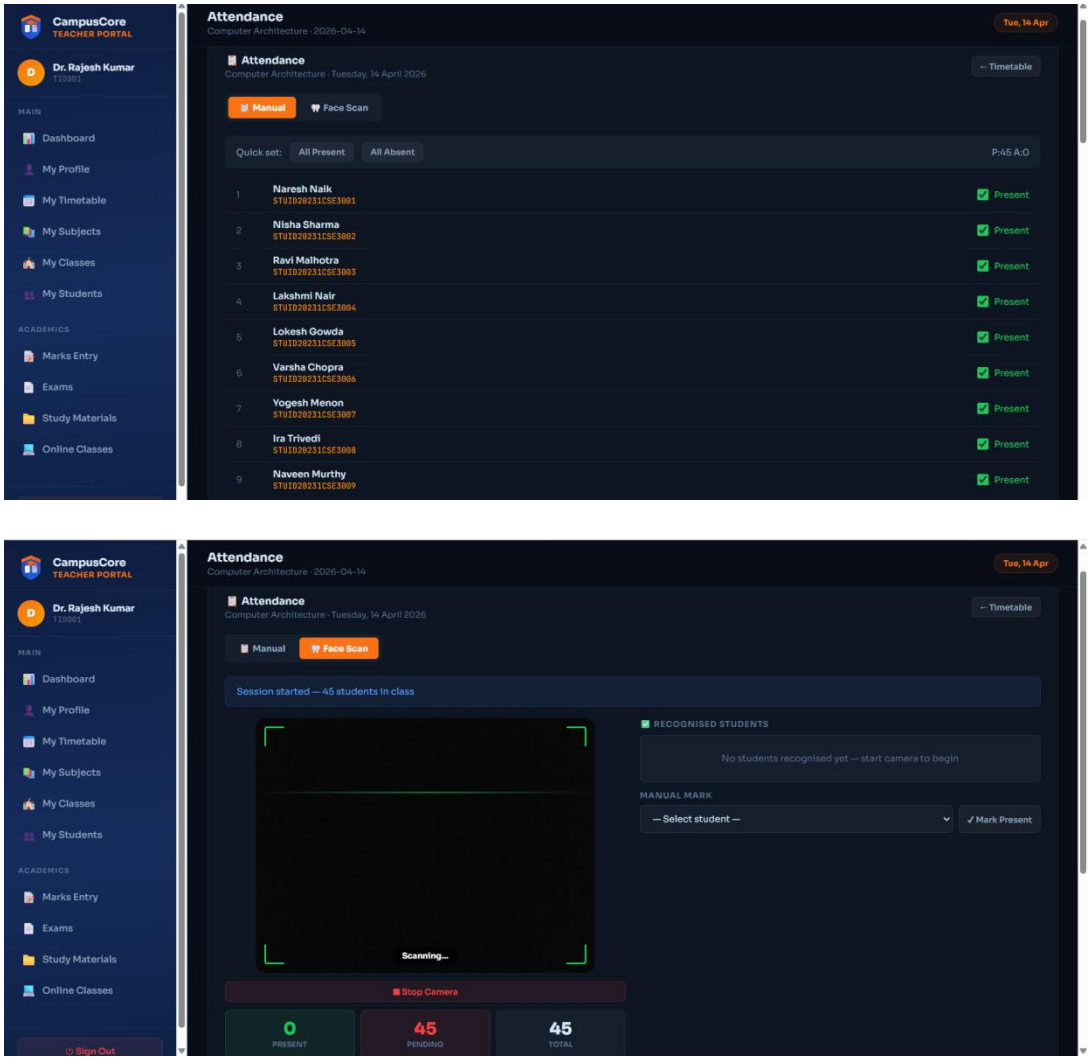


Figure 7.3: Attendance Flow Diagram showing Manual attendance list submission and Face Recognition webcam session with real-time student identification

CHAPTER 8

SOCIAL, LEGAL, ETHICAL, AND SUSTAINABILITY ASPECTS

8.1 Social Aspects

The deployment of a comprehensive digital ERP system in a university environment carries significant social implications. For students, a transparent and accessible portal that shows real-time attendance, marks, and fee status eliminates information asymmetry — a historically common grievance in Indian colleges where students often had to visit administrative offices or rely on notice boards for information that should be readily available to them. CampusCore's notification module enables institutions to push timely alerts about exam schedules, fee deadlines, and attendance warnings directly to student accounts.

For administrative staff, digitisation reduces the burden of manual record-keeping, minimises paperwork, and accelerates processes such as generating attendance reports or compiling semester-end marks. This efficiency gain translates to staff having more time for student-facing responsibilities. For faculty, tools like online examination creation and study material upload reduce the logistical overhead of assessment, enabling more time to be invested in teaching quality.

CampusCore also has an indirect social equity benefit: by being open-source and deployable on low-cost hardware with SQLite, it is accessible to institutions that cannot afford commercial ERP platforms. This democratises access to digital administration infrastructure across different socioeconomic tiers of higher education.

8.2 Legal Aspects

The handling of student data in CampusCore is subject to multiple legal frameworks in India. The Information Technology Act, 2000 (as amended in 2008) under Section 43A imposes liability on companies that handle sensitive personal data negligently.

The Digital Personal Data Protection Act, 2023 (DPDP Act) — India's first comprehensive data protection legislation — establishes obligations for data fiduciaries to collect data only for specified purposes, retain it for no longer than necessary, and implement reasonable security safeguards.

CampusCore addresses these obligations through several design choices. Passwords are never stored in plain text; all password fields use cryptographic hash functions (SHA-256 or bcrypt). Date of birth data, used as an initial password, is not exposed in any API response after authentication. The system does not collect more personal information than is necessary for academic administration. Activity logs capture user actions for accountability but are not designed to profile users beyond operational necessity.

In the context of institutional deployment, universities using CampusCore would be classified as Data Fiduciaries under the DPDP Act and would need to ensure compliance through a Data Processing Agreement governing how the software is operated and how data is retained and deleted. This is a governance responsibility of the deploying institution rather than the software itself.

8.3 Ethical Aspects

Several ethical considerations arise in the design and use of a university ERP. First, the Manual attendance system provides a straightforward method for teachers to directly record student presence. However, the Face Recognition-based attendance feature raises significant privacy concerns: facial recognition constitutes processing of biometric data and must comply with applicable data protection regulations. Students should be informed and give explicit consent before enrolment. The current implementation stores facial encodings as binary blobs in the StudentFaceData table and processes them only on the server. The system provides a fallback to Manual attendance in cases where face recognition is unavailable or a student has not registered their face.

Second, the face-recognition-based attendance feature (partially implemented in face_recognition_helper.py) raises significant ethical concerns. Facial recognition constitutes processing of biometric data — a category of sensitive personal information under the DPDP Act.

Its use must be disclosed to students, and informed consent must be obtained before enrolment. The current implementation stores facial encodings as binary blobs in the StudentFaceData table and processes them only on the server side; no images are persisted, and encoding data should be deleted when a student graduates or withdraws.

Third, the exam module's online MCQ functionality, if used for summative assessments, requires proctoring mechanisms to ensure academic integrity. Without such safeguards, online exams may incentivise misconduct. CampusCore's exam module is designed primarily for formative continuous assessment (CA1–CA5) rather than high-stakes examinations, which partially mitigates this concern.

8.4 Sustainability Aspects

Digital transformation of university administration contributes to sustainability across multiple dimensions. By replacing physical registers, printed attendance sheets, and paper-based fee receipts with a digital platform, CampusCore reduces paper consumption. At an institution with 2,000 students across multiple departments, the elimination of daily attendance sheets alone could save several reams of paper per month.

From an energy perspective, the application is designed to be lightweight. The Flask backend consumes minimal server resources, and the frontend avoids heavy JavaScript frameworks that would increase CPU usage on client devices. The Railway cloud deployment used for production hosting operates on shared, efficiently utilised infrastructure, which has a lower per-application carbon footprint than dedicated physical servers.

The database's multi-backend support means institutions can start with SQLite on a modest server and migrate to a more robust database only when scale demands it, avoiding premature investment in expensive infrastructure.

8.5 Safety Aspects

From a software safety perspective, the most critical concern is the security of student data. CampusCore implements several defensive programming practices: all database queries use parameterised statements (no string concatenation) to prevent SQL injection; file uploads are validated for MIME type and size to prevent malicious file execution;

JWT expiry ensures that stolen tokens have a limited attack window; and error handlers return generic messages to clients while logging detailed errors server-side to avoid information leakage.

CHAPTER 9

CONCLUSION

9.1 Summary of Work

This internship, conducted at Rail Wheel Factory (RWF), Yelahanka, Bengaluru, culminated in the development of CampusCore — a full-stack, web-based University ERP System designed to centralise and streamline academic administration. Over twelve weeks, the system progressed from initial database schema design through to a working application with over forty RESTful API endpoints, four role-specific frontend dashboards, and comprehensive coverage of core ERP modules including student management, teacher management, attendance tracking, fee management, online examinations, study materials, and timetabling.

The technology stack — Python Flask, HTML/CSS/JavaScript, and SQL Server with SQLite/MySQL fallbacks — was deliberately chosen to balance development speed, portability, and alignment with industry-standard practices. The application was version-controlled on GitHub throughout the internship and configured for deployment on Railway cloud, demonstrating real-world deployment readiness.

9.2 Achievement vs. Objectives

Objective	Achievement Status
O1: Normalised database schema	Fully achieved — 22-table schema in 3NF with PK/FK constraints across all entities
O2: Secure Flask REST API with JWT	Fully achieved — JWT auth, brute-force protection, role-based access on all endpoints
O3: Role-based frontend dashboards	Achieved — Admin, Teacher, Student, and Change-Password pages fully integrated
O4: Manual & Face Recognition attendance system	Achieved — Time-limited token generation and scan-based attendance marking functional

9.3 Limitations

Despite the progress achieved, several limitations remain. The face-recognition-based attendance feature is partially implemented — the helper module exists but the API integration for live webcam feed processing was not completed within the internship timeframe. The reporting module, which would generate downloadable PDF reports for attendance, marks, and fee collection, was listed as future work. The frontend, while functional, lacks a systematic responsive design framework, which means some dashboard views may not render optimally on small mobile screens. The brute-force protection mechanism is in-memory, meaning it resets when the Flask server restarts and is not suitable for multi-process production deployments without a shared cache like Redis.

9.4 Future Improvements

Several enhancements are envisioned for CampusCore's continued development. The face-recognition attendance pipeline should be completed, with proper consent mechanisms and biometric data governance. A reporting module using a library such as ReportLab or WeasyPrint would enable PDF generation for all key administrative reports. Integration with a payment gateway (such as Razorpay or PayU) would enable actual online fee transactions rather than manual payment recording. A mobile application built on React Native or Flutter would significantly improve student accessibility. Finally, an analytics dashboard with visualisations for attendance trends, fee collection rates, and exam performance would provide institutional leadership with actionable insights.

REFERENCES

1. Abugabah, A. and Sanzogni, L. (2010) 'Enterprise resource planning (ERP) system in higher education: A literature review and implications', International Journal of Human and Social Sciences, 5(6), pp. 395–399.
2. Bansal, R. and Goyal, N. (2015) 'Automated fee management systems for engineering institutions', International Journal of Computer Applications, 119(15), pp. 21–25.
3. Bayer, M. (2012) SQLAlchemy: The Database Toolkit for Python. O'Reilly Media: Sebastopol, CA.
4. Codd, E.F. (1970) 'A relational model of data for large shared data banks', Communications of the ACM, 13(6), pp. 377–387.
5. Chen, P.P.S. (1976) 'The entity-relationship model—toward a unified view of data', ACM Transactions on Database Systems, 1(1), pp. 9–36.
6. Davenport, T.H. (1998) 'Putting the enterprise into the enterprise system', Harvard Business Review, 76(4), pp. 121–131.
7. Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine.
8. Fowler, M. (2002) Patterns of Enterprise Application Architecture. Addison-Wesley: Reading, MA.
9. Grinberg, M. (2018) Flask Web Development: Developing Web Applications with Python. 2nd edn. O'Reilly Media: Sebastopol, CA.
10. Jain, S. and Shanbhag, D. (2012) 'Addressing security vulnerabilities in web applications', International Journal of Computer Science and Information Security, 10(12), pp. 45–52.
11. Johansson, B. and Bjorn-Andersen, N. (2015) 'Open source ERP adoption in educational institutions', Journal of Information Technology Education: Research, 14, pp. 127–148.

12. Kaur, H. and Singh, G. (2019) 'A review of automated attendance management systems', International Journal of Computer Science and Engineering, 7(2), pp. 318–323.

13. Ministry of Education, Government of India (2022) All India Survey on Higher Education 2021–22. New Delhi: Department of Higher Education.

14. NASSCOM (2022) Indian EdTech: An Overview of the Landscape. NASSCOM: New Delhi.

15. Nielsen, J. (1993) Usability Engineering. Academic Press: Boston, MA.

16. OWASP (2021) OWASP Top Ten 2021. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 15 March 2026).

17. Pollock, N. and Cornford, J. (2004) 'ERP systems and the university as a 'unique' organisation', Information Technology and People, 17(1), pp. 31–52.

18. Richardson, L. and Ruby, S. (2007) RESTful Web Services. O'Reilly Media: Sebastopol, CA.

19. Ronacher, A. (2010) Flask Documentation 3.x. Available at: https://flask.palletsprojects.com/ (Accessed: 15 March 2026).

20. Wroblewski, L. (2011) Mobile First. A Book Apart: New York, NY.

APPENDIX

Appendix A: Screenshot Placeholders

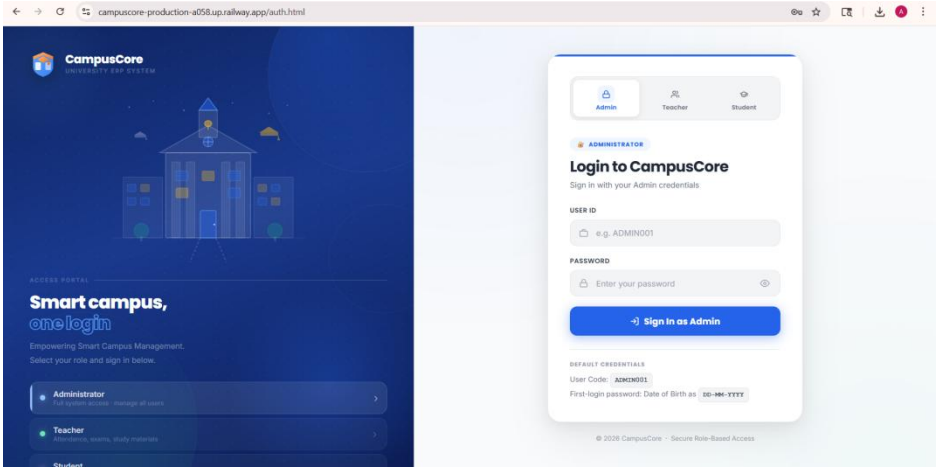


Fig A.1: Login page (auth.html) showing user code and password fields with the CampusCore branding

FILE: auth.html — CampusCore Login Page

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>CampusCore — Sign In</title>

  <!-- Role-based CSS: Admin=Blue, Teacher=Green, Student=Purple -->

  <style>

    body { font-family: 'Inter'; display: flex; }

    /* Left decorative panel */

    .panel { flex: 0 0 44%; background: linear-gradient(160deg, #0f2460, #1a3a8f); }



    /* Right login card */

    .stage { flex: 1; display: flex; align-items: center; justify-content: center; }

    .card  { background: #fff; border-radius: 22px; padding: 2rem; max-width: 420px; }
```

```
    /* Color theme switches based on selected role */
    body[data-role="Admin"]   { --accent: #2563EB; }
    body[data-role="Teacher"] { --accent: #10B981; }
    body[data-role="Student"] { --accent: #7c3aed; }
    .card-accent { height: 3.5px; background: var(--accent); }
    .field input:focus { border-color: var(--accent); }
  </style>
</head>
<body data-role="Admin">
  <!-- Left Panel: Logo + Campus Illustration + Role Pills -->
  <aside class="panel">
    <div class="brand">CampusCore</div>
    <div class="campus-art"> <!-- SVG university building --> </div>
    <div class="role-pills">
      <div onclick="selectRole('Admin')">   Administrator </div>
      <div onclick="selectRole('Teacher')"> Teacher       </div>
      <div onclick="selectRole('Student')"> Student       </div>
    </div>
  </aside>
  <!-- Right Panel: Login Form -->
  <main class="stage">
    <div class="card">
      <div class="card-accent"></div>
      <!-- Role Tabs -->
      <div class="role-tabs">
        <button onclick="selectRole('Admin')">   Admin   </button>
```

```
      <button onclick="selectRole('Teacher')"> Teacher </button>
      <button onclick="selectRole('Student')"> Student </button>
    </div>
    <!-- Dynamic badge + title update on role switch -->
    <div id="roleBadge">Administrator</div>
    <div id="cardTitle">Login to CampusCore</div>
    <!-- Alerts & wrong-role hint -->
    <div id="alertBox"></div>
    <div id="switchHint"></div>
    <!-- Login Form -->
    <form onsubmit="handleLogin(event)">
      <input id="userCode" type="text"     placeholder="User ID"  required>
      <input id="password" type="password" placeholder="Password" required>
      <button type="submit" id="loginBtn">Sign In</button>
    </form>
  </div>
 </main>
</body>
<script>
 const API = localStorage.getItem('api_base') || window.location.origin;
 let selectedRole = 'Admin';
 // Role config: controls badge, button color, placeholder & redirect
 const ROLES = {
  Admin:   { redirect: 'admin.html'   },
  Teacher: { redirect: 'teacher.html' },
  Student: { redirect: 'student.html' }
 };
```

```
// Switch role → updates CSS theme, tabs, badge, button, hints
function selectRole(role) {
  selectedRole = role;
  document.body.setAttribute('data-role', role); // triggers CSS color change
}
// Main login handler
async function handleLogin(e) {
  e.preventDefault();
  const userCode = document.getElementById('userCode').value.trim();
  const password = document.getElementById('password').value;


  const res  = await fetch(API + '/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ userCode, password, loginAs: selectedRole })
  });
  const data = await res.json();
  if (!data.success) {
    if (data.actualRole) showSwitchHint(data.actualRole); // wrong role selected
    else showAlert('error', data.error);                  // invalid credentials
    return;
  }
  // Save session → redirect to role dashboard
  localStorage.setItem('token', data.token);
  localStorage.setItem('user',  JSON.stringify(data.user));
  location.href = data.user.isFirstLogin
    ? 'change_password.html'          // force password change on first login
```

```
    : ROLES[selectedRole].redirect;   // go to role dashboard
  }
  // Auto-redirect if already logged in (validates JWT expiry)
  (function checkSession() {
    const token = localStorage.getItem('token');
    const user  = JSON.parse(localStorage.getItem('user') || 'null');
    if (!token || !user) return;
    // decode JWT → check exp → redirect or clear storage
  })();
</script>
</html>
```

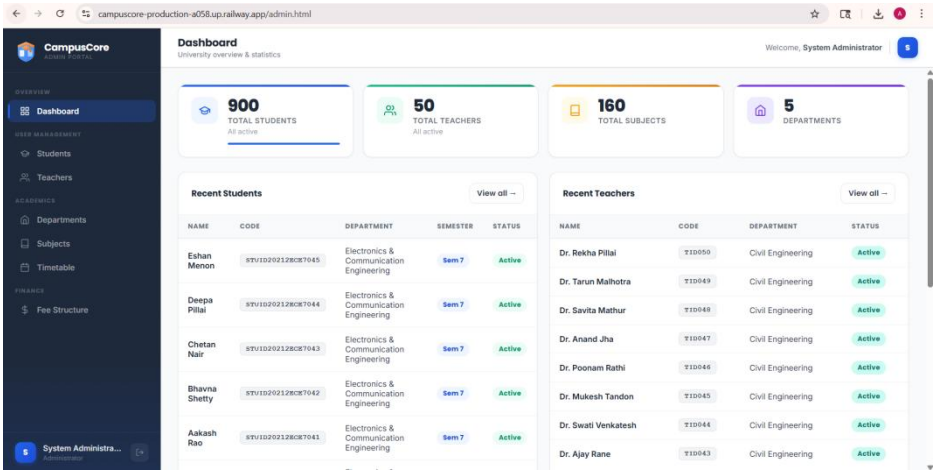


Fig A.2: Admin Dashboard showing student count , teacher count, and navigation panel

FILE: admin.html — CampusCore Admin Dashboard

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CampusCore — Admin Dashboard</title>
  <style>
```

```
    /* Layout: fixed sidebar + scrollable main content */
    body    { display: flex; background: #F8FAFC; }
    .sidebar { width: 260px; position: fixed; height: 100vh; background: #1E293B; }
    main     { margin-left: 260px; flex: 1; }
    .topbar  { position: sticky; top: 0; height: 64px; background: #fff; border-bottom: 1px solid
#E5E7EB; }

    /* Pages shown/hidden via JS */
    .page       { display: none; }
    .page.active { display: block; }

    /* Stat cards, data tables, modals — standard card layout */
    .card   { background: #fff; border-radius: 14px; border: 1px solid #E5E7EB; }
    .modal-backdrop       { display: none; position: fixed; inset: 0; }
    .modal-backdrop.open   { display: flex; }
  </style>
</head>
<body>
  <!-- SIDEBAR: Navigation -->
  <aside class="sidebar">
    <div>CampusCore — Admin Portal</div>
    <nav>
      <!-- Each item calls showPage() to switch the active page -->
      <div onclick="showPage('dashboard')"> Dashboard   </div>
      <div onclick="showPage('students')">  Students    </div>
      <div onclick="showPage('teachers')">  Teachers    </div>
      <div onclick="showPage('departments')">Departments </div>
```

```
    <div onclick="showPage('subjects')"> Subjects    </div>
    <div onclick="showPage('timetable')"> Timetable   </div>
    <div onclick="showPage('fees')">      Fee Structure</div>
  </nav>
  <button onclick="logout()">Logout</button>
</aside>
<!-- MAIN -->
<main>
  <div class="topbar">
    <span id="pageTitle">Dashboard</span>
    <span>Welcome, <b id="topName">Admin</b></span>
  </div>
  <div class="content">
    <!-- Each section is a .page div; only one is .active at a time -->
    <div class="page active" id="page-dashboard">
      <!-- Stat cards: Total Students, Teachers, Subjects, Departments -->
      <!-- Recent Students table + Recent Teachers table -->
      <!-- Students by Department + Quick Action buttons -->
    </div>
    <div class="page" id="page-students">
      <!-- Search bar + "Add Student" button -->
      <!-- Table: Name | Code | Email | Phone | Dept | Sem | Status | Actions -->
      <!-- Actions: Edit | Reset Password | Activate/Deactivate -->
    </div>

    <div class="page" id="page-teachers">
      <!-- Same pattern as Students page -->
```

```
    </div>
    <div class="page" id="page-departments">
      <!-- Table: Dept Name | Code | Semesters | Students | Teachers | Delete -->
    </div>
    <div class="page" id="page-subjects">
      <!-- Table: Subject | Code | Credits | Semester | Dept | Type | Delete -->
    </div>
    <div class="page" id="page-timetable">
      <!-- Filter by Dept + Semester -->
      <!-- Table: Dept | Sem | Subject | Teacher | Day | Period | Time | Room -->
    </div>
    <div class="page" id="page-fees">
      <!-- Table: Dept | Sem | Year | Tuition | Lab | Library | Sports | Total -->
    </div>
  </div>
</main>
<!-- MODALS (one per action: Add/Edit Student, Teacher, Dept, Subject, Timetable, Fee) -->
<div class="modal-backdrop" id="addStudentModal"> ... </div>
<div class="modal-backdrop" id="editStudentModal"> ... </div>
<div class="modal-backdrop" id="resetPwModal"> ... </div>
<!-- + addTeacher, editTeacher, addDept, addSubject, addTimetable, addFee -->
<script>
const API   = localStorage.getItem('api_base') || window.location.origin;
let TOKEN   = '';
let depts=[], teacherList=[], subjectList=[], allStudents=[], allTeachers=[];
// ── Init: verify Admin auth, load all data
function init() {
```

```
  TOKEN = localStorage.getItem('token') || '';
  const USER = JSON.parse(localStorage.getItem('user') || '{}');
  if (!TOKEN || USER.userType !== 'Admin') { location.href = 'auth.html'; return; }
  loadDashboard(); loadDepts(); loadStudentsData(); loadTeachersData();
}
// ── API helper: attaches Bearer token to every request
async function api(url, opts = {}) {
  opts.headers = { 'Authorization': 'Bearer ' + TOKEN, 'Content-Type': 'application/json' };
  const r = await fetch(API + url, opts);
  if (r.status === 401) { logout(); return null; }
  return await r.json();
}
// ── Page switcher
function showPage(id) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.getElementById('page-' + id).classList.add('active');
}
// ── Dashboard: load stats + recent records
async function loadDashboard() {
  const d = await api('/api/admin/dashboard');
  document.getElementById('s-students').textContent = d.stats.totalStudents;
  document.getElementById('s-teachers').textContent = d.stats.totalTeachers;
  // … render recentStudents and recentTeachers tables
}


// ── Students CRUD
async function loadStudents()       { /* GET  /api/admin/students       → render table */ }
```

```
 async function addStudent()        { /* POST /api/admin/students       → reload      */ }

 async function saveStudent()       { /* PUT  /api/admin/students/:id   → reload      */ }

 async function toggleStudent(id, active) { /* POST /api/admin/students/:id/toggle-active */ }

 // ── Teachers CRUD (same pattern as Students)

 async function loadTeachers()      { /* GET  /api/admin/teachers       */ }

 async function addTeacher()        { /* POST /api/admin/teachers       */ }

 async function saveTeacher()       { /* PUT  /api/admin/teachers/:id   */ }

 async function toggleTeacher(id, active) { /* POST /api/admin/teachers/:id/toggle-active */ }

 // ── Password reset

 async function doResetPassword() {

   const id = document.getElementById('reset-user-id').value;

   const pw = document.getElementById('reset-new-pw').value.trim();

   // POST /api/admin/reset-password/:id  →  { newPassword } or blank = reset to DOB

   const d = await api(`/api/admin/reset-password/${id}`, { method:'POST', body: JSON.stringify(pw ?
{newPassword:pw} : {}) });

   if (d?.success) toast('Password reset: ' + d.newPassword);

 }

 // ── Departments, Subjects, Timetable, Fees (same GET/POST/DELETE pattern)

 async function loadDepts()     { /* GET /api/admin/departments    → fill table + all dropdowns */ }

 async function addDept()       { /* POST /api/admin/departments   */ }

 async function deleteDept(id)  { /* DELETE /api/admin/departments/:id */ }

 async function loadSubjects()      { /* GET /api/admin/subjects   */ }

 async function addSubject()        { /* POST /api/admin/subjects  */ }

 async function deleteSubject(id)   { /* DELETE /api/admin/subjects/:id */ }


 async function loadTimetable()     { /* GET /api/admin/timetable?departmentId=&semester= */ }

 async function addTimetable()      { /* POST /api/admin/timetable  */ }
```

```
  async function deleteTimetable(id) { /* DELETE /api/admin/timetable/:id */ }

  async function loadFees()   { /* GET /api/admin/fee-structure  */ }

  async function addFee()     { /* POST /api/admin/fee-structure */ }

  // ── Helpers

  function openModal(id)  { document.getElementById(id).classList.add('open');    }

  function closeModal(id) { document.getElementById(id).classList.remove('open'); }

  function toast(msg, type='success') { /* show temporary notification */ }

  function logout() { localStorage.clear(); location.href = 'auth.html'; }

  init();

</script>

</body>

</html>
```

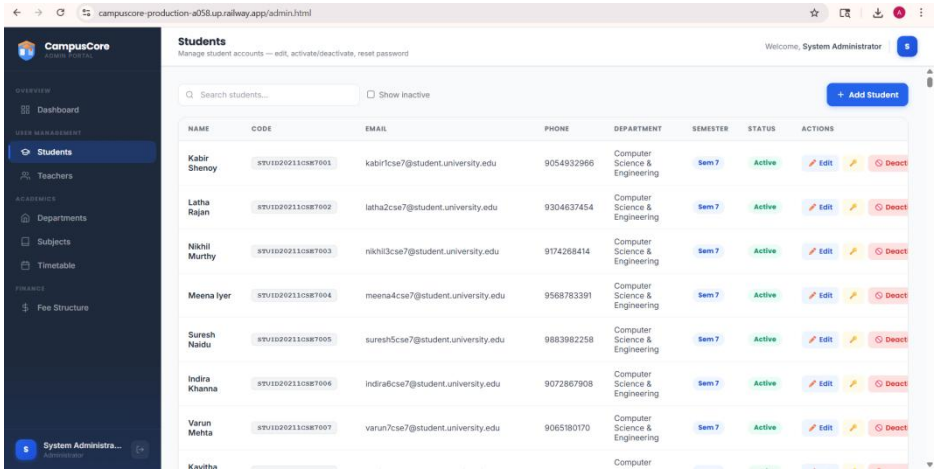


Fig A.3: Student Management table in admin panel with columns for Roll Number, Name, Department, Semester, Email, and Action buttons

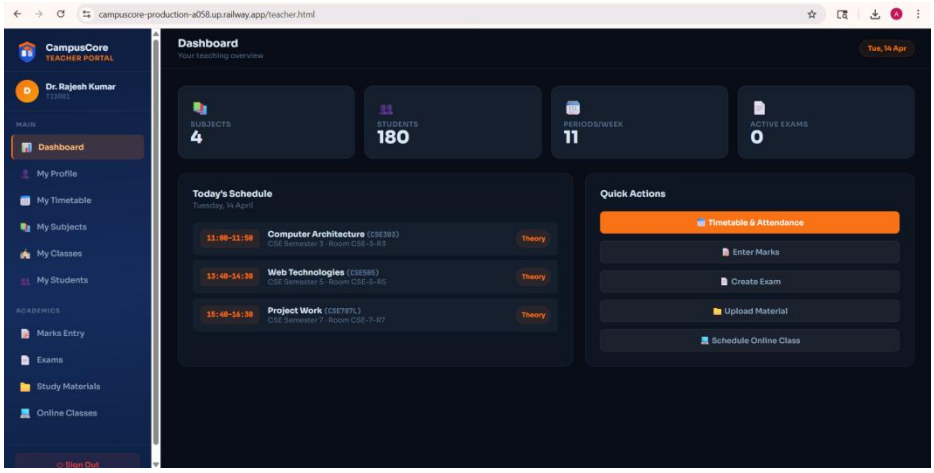


Fig A.4: Teacher Dashboard showing today's timetable and subject list

FILE: teacher.html — CampusCore Teacher Portal

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>CampusCore — Teacher Portal</title>

  <style>

    /* Dark theme layout */

    body    { background: #080d18; color: #dce8f5; display: flex; }

    .sidebar { width: 260px; position: fixed; height: 100vh;

               background: linear-gradient(175deg, #0f1e40, #162d5e); }

    .main   { margin-left: 260px; flex: 1; }

    .topbar { position: sticky; top: 0; background: rgba(8,13,24,.92);

               backdrop-filter: blur(12px); }

    /* Views shown/hidden via JS */

    .view        { display: none; }

    .view.active { display: block; }

    /* Modal overlay */
```

```
    .modal-overlay      { display: none; position: fixed; inset: 0; }
    .modal-overlay.open { display: flex; }
  </style>
</head>
<body>
  <!-- SIDEBAR: Navigation -->
  <aside class="sidebar">
    <div>CampusCore — Teacher Portal</div>
    <div id="sbName">Loading…</div>  <!-- filled from /api/teacher/profile -->
    <nav>
      <!-- Each nav item calls go() to switch the active view -->
      <div onclick="go('dashboard')">      Dashboard      </div>
      <div onclick="go('profile')">        My Profile     </div>
      <div onclick="go('timetable')">      My Timetable   </div>
      <div onclick="go('subjects')">       My Subjects    </div>
      <div onclick="go('classes')">        My Classes     </div>
      <div onclick="go('students')">       My Students    </div>
      <div onclick="go('marks')">          Marks Entry    </div>
      <div onclick="go('exams')">          Exams          </div>
      <div onclick="go('materials')">      Study Materials  </div>
      <div onclick="go('online')">         Online Classes   </div>
    </nav>
    <button onclick="logout()">Sign Out</button>
  </aside>


  <!-- MAIN -->
  <main class="main">
```

```
<div class="topbar">
  <h2 id="pageTitle">Dashboard</h2>
</div>
<div class="content">
  <!-- One .view div per section — only one active at a time -->
  <div class="view active" id="view-dashboard">
    <!-- Stat cards: Subjects | Students | Periods/Week | Active Exams -->
    <!-- Today's schedule list + Quick Action buttons -->
  </div>
  <div class="view" id="view-profile">
    <!-- Teacher info: Name, Code, Dept, Email, Phone, DOB -->
  </div>
  <div class="view" id="view-timetable">
    <!-- Weekly grid: Mon–Fri × Period slots -->
    <!-- Each cell is clickable to open Attendance view for that class -->
    <!-- Color coded: Green=Done, Red=Missed, Amber=Today, Blue=Upcoming -->
  </div>
  <div class="view" id="view-subjects">
    <!-- Table: Code | Name | Credits | Type (Theory/Lab) | Sem | Dept -->
  </div>
  <div class="view" id="view-classes">
    <!-- Card grid: Class name | Dept | Subject count | Student count -->
  </div>
  <div class="view" id="view-students">
    <!-- Filter by class + search by name/ID -->
    <!-- Table: # | Student ID | Name | Semester | Gender | Dept | Email -->
  </div>
```

```
    <div class="view" id="view-attendance">
      <!-- Context: Subject + Date from timetable cell click -->
      <!-- Tab 1: Manual — checkbox roster (Present/Absent per student) -->
      <!-- Tab 2: Face Scan — webcam + live recognition + manual override -->
      <!-- History table at bottom -->
    </div>
    <div class="view" id="view-marks">
      <!-- Step 1: Select Subject + Exam Type (CA1–CA5, Midterm, Endterm) -->
      <!-- Step 2: Editable roster — number input per student (0–max) -->
      <!-- Saved marks shown in read-only mode -->
    </div>
    <div class="view" id="view-exams">
      <!-- Table of exams + "Add Question" button per row -->
    </div>
    <div class="view" id="view-materials">
      <!-- Table of uploaded resources + file/URL open link -->
    </div>
    <div class="view" id="view-online">
      <!-- Table of scheduled online classes + Join link -->
    </div>
  </div>
</main>
<!-- MODALS -->
<!-- createExamMdl   : Subject, Exam Name, Type, Marks, Date, Start/End Time -->
<!-- addQuestionMdl  : Question text, Type (MCQ/TF/Short), Options, Answer -->
<!-- addMatMdl       : Subject, Title, Description, File URL or Upload -->
<!-- addOCMdl        : Subject, Topic, Date, Start/End Time, Meeting Link -->
```

```
<script>
 const API   = localStorage.getItem('api_base') || window.location.origin;
 let token   = '';
 let mySubjects = [];
 // ── Init: verify auth, load sidebar profile, pre-load subjects
 async function init() {
  token = localStorage.getItem('accessToken') || localStorage.getItem('token') || '';
  if (!token) { location.href = 'auth.html'; return; }
  const pr = await api('/api/teacher/profile');
  if (pr.success) {
   document.getElementById('sbName').textContent = pr.profile.FullName;
  }
  await loadSubjectsIntoSelects();  // populate all subject dropdowns
  loadDash();
 }
 // ── API helper: attaches Bearer token
 async function api(ep, opts = {}) {
  const r = await fetch(API + ep, {
   headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + token },
   ...opts
  });
  if (r.status === 401) { localStorage.clear(); location.href = 'auth.html'; }
  return await r.json().catch(() => ({}));
 }


 // ── View switcher
 function go(view) {
```

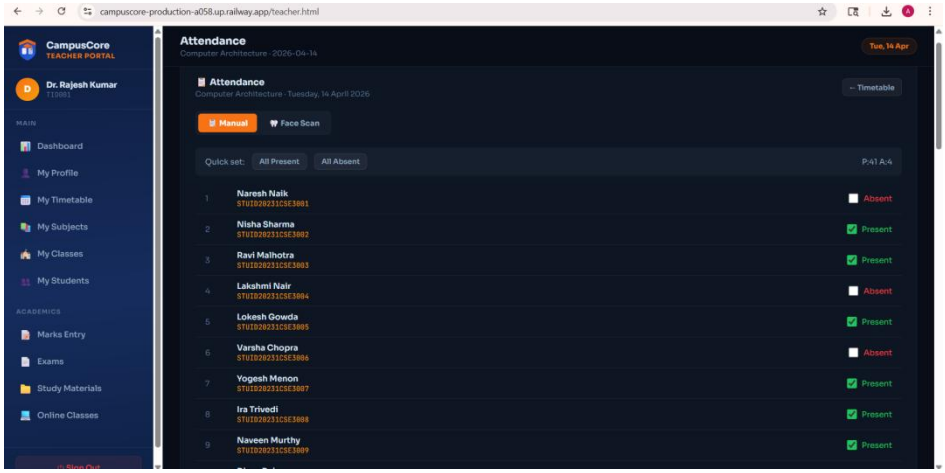
```
  document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
  document.getElementById('view-' + view).classList.add('active');
  loadView(view);
}
function loadView(v) {
  const map = {
    dashboard: loadDash,      profile:   loadProfile,
    timetable: loadTT,        subjects:  loadSubjs,
    classes:   loadClasses,   students:  loadStudents,
    attendance: loadAttView,  marks:     loadMarksView,
    exams:     loadExams,     materials: loadMats,
    online:    loadOC
  };
  if (map[v]) map[v]();
}
// ── Subjects: pre-load into all <select> dropdowns
async function loadSubjectsIntoSelects() {
  const d = await api('/api/teacher/subjects');
  mySubjects = d.subjects || [];
  const opts = mySubjects.map(s =>
    `<option value="${s.SubjectID}">${s.SubjectName} (${s.SubjectCode}) - Sem ${s.Semester}</option>`
  ).join('');
  ['marksSubject','examSubject','matSubject','ocSubject'].forEach(id => {
    const el = document.getElementById(id);
    if (el) el.innerHTML = '<option value="">-- Select Subject --</option>' + opts;
  });
```

```
}
// ── Dashboard
async function loadDash() {
  const d = await api('/api/teacher/dashboard');
  // fill stat cards + today's schedule from d.stats.todaysSchedule
}
// ── Timetable: Mon–Fri grid; clicking a cell opens attendance
async function loadTT() {
  const d = await api('/api/teacher/timetable');
  // build grid, color-code cells by date vs today
  // cell onclick → openAttFromTT(subjectId, name, date, deptId, semester, alreadyDone)
}
// ── Attendance (Manual + Face Recognition)
let attRosterData = [], _attSubjId, _attDate, _attDeptId, _attSem, _attLocked;
let _faceStream, _faceScanInterval, _faceSessionToken, _faceRecognised = new Map();
function openAttFromTT(subjectId, name, date, deptId, sem, locked) {
  _attSubjId = subjectId; _attDate = date;
  _attDeptId = deptId;   _attSem  = sem; _attLocked = locked;
  go('attendance');
}
async function loadAttView() {
  // GET /api/teacher/students/by-class/:deptId/:sem → build checkbox roster
}
async function saveAttendance() {
  // POST /api/teacher/attendance/submit
  // body: { subjectId, attendanceDate, attendance: [{studentId, status}] }
}
```

```
// Face Recognition flow:
async function startFaceCamera() {
  // 1. getUserMedia({ video }) → show webcam
  // 2. POST /api/teacher/attendance/face/start → get sessionToken
  // 3. setInterval: captureFaceFrame() every 2s
}
async function captureFaceFrame() {
  // canvas.drawImage(video) → toDataURL('image/jpeg')
  // POST /api/teacher/attendance/face/capture
  //   body: { sessionToken, subjectId, attendanceDate, image }
  // Response: { newlyRecognised: [{studentId, fullName, rollNumber}] }
  // → add matched students to recognised list
}
async function finaliseFaceAttendance() {
  // POST /api/teacher/attendance/mark-absent
  //   body: { subjectId, attendanceDate, sessionToken }
  // Marks all unrecognised students as Absent
}
// ── Marks Entry
const MARKS_MAX = { CA1:10, CA2:10, CA3:10, CA4:10, CA5:10, Midterm:50, Endterm:100 };
async function loadMarksRoster() {
  // GET /api/teacher/marks/students?subjectId=X
  // Render editable number inputs (0 – MARKS_MAX[examType])
}


async function saveMarks() {
  // POST /api/teacher/marks
```

```
  // body: { subjectId, marks: [{studentId, CA1, CA2, … Midterm, Endterm}] }
}

// ── Exams
async function createExam() {
  // POST /api/teacher/exams
  // body: { subjectId, examName, examType, totalMarks, duration, examDate, startTime, endTime }
}
async function submitAddQuestion() {
  // POST /api/teacher/exam-questions
  // body: { examId, questionText, questionType, marks, correctAnswer, optionA–D }
}
// ── Study Materials
async function saveMaterial() {
  // If file selected: POST /api/teacher/materials/upload (multipart, max 50 MB)
  // Then: POST /api/teacher/materials
  //   body: { subjectId, title, description, filePath, fileType }
}
// ── Online Classes
async function scheduleOC() {
  // POST /api/teacher/online-class
  // body: { subjectId, topic, scheduledDate, startTime, endTime, meetingLink }
}



// ── Helpers
function openMdl(id)  { document.getElementById(id).classList.add('open');    }
```

```
  function closeMdl(id) { document.getElementById(id).classList.remove('open'); }

  function toast(msg, type) { /* append temp notification div */ }

  function logout() { localStorage.clear(); location.href = 'auth.html'; }

  init();

</script>

</body>

</html>
```



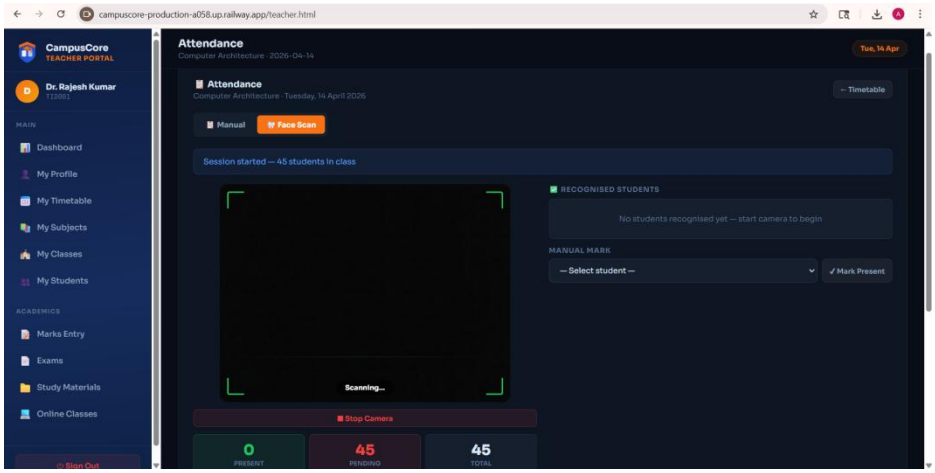


Fig A.5: Teacher attendance panel showing Manual mode (student list with Present/Absent toggles) and Face Scan mode (live webcam feed with recognised student overlay)

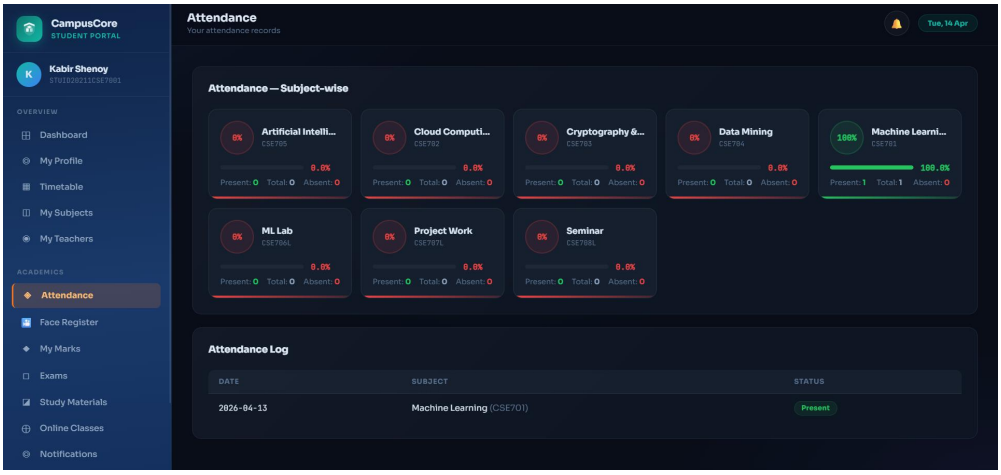


Fig A.6: Student Portal – Attendance tab showing subject-wise percentage with colour indicators

FILE: student.html — CampusCore Student Portal

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>CampusCore — Student Portal</title>

  <style>

    /* Dark theme layout */

    body    { background: #080d18; color: #dce8f5; display: flex; }

    .sidebar { width: 260px; position: fixed; height: 100vh;

               background: linear-gradient(175deg, #0f2447, #1a3a6b); }

    .main   { margin-left: 260px; flex: 1; }

    .topbar { position: sticky; top: 0; background: rgba(8,13,24,.92);

               backdrop-filter: blur(12px); }

    /* Views shown/hidden via JS */

    .view        { display: none; }

    .view.active { display: block; }

    /* Timetable slot colors by attendance status */
```

```
  .tt-slot.att-present  { border-color: rgba(34,197,94,.4);  }  /* green  */
  .tt-slot.att-absent   { border-color: rgba(239,68,68,.45); }  /* red    */
  .tt-slot.att-no-att   { border-color: rgba(245,158,11,.4); }  /* amber  */
  .tt-slot.att-upcoming { border-color: rgba(59,130,246,.35);}  /* blue   */
  .tt-slot.att-lab      { border-color: rgba(168,85,247,.3); }  /* purple */
 </style>
</head>
<body>
 <!-- SIDEBAR: Navigation -->
 <aside class="sidebar">
  <div>CampusCore — Student Portal</div>
  <div id="sbName">Loading…</div>  <!-- filled from /api/student/profile -->
  <nav>
   <!-- Overview -->
   <div onclick="go('dashboard')">  ⊞  Dashboard      </div>
   <div onclick="go('profile')">    ◎ My Profile      </div>
   <div onclick="go('timetable')">  ■  Timetable      </div>
   <div onclick="go('subjects')">   ◫  My Subjects     </div>
   <div onclick="go('teachers')">   ◉  My Teachers     </div>
   <!-- Academics -->
   <div onclick="go('attendance')"> ◈  Attendance      </div>
   <div onclick="go('scan')">          Face Register   </div>
   <div onclick="go('marks')">      ◆ My Marks        </div>
   <div onclick="go('exams')">      □ Exams           </div>
   <div onclick="go('materials')">  ◪  Study Materials  </div>
   <div onclick="go('online')">     ⊕ Online Classes   </div>
   <div onclick="go('notifications')">◎ Notifications  </div>
```

```
    <div onclick="go('fees')">      ⊛  Fee Details     </div>
  </nav>
  <button onclick="logout()">Sign Out</button>
</aside>
<!-- MAIN -->
<main class="main">
  <div class="topbar">
    <h2 id="pageTitle">Dashboard</h2>
  </div>
  <div class="content">
    <div class="view active" id="view-dashboard">
      <!-- Stat cards: Subjects | Attendance% | Semester | Pending Exams | Notifications -->
      <!-- Today's Classes list + Attendance Summary with progress bars -->
    </div>
    <div class="view" id="view-profile">
      <!-- Grid: Name, Roll No, Email, Phone, Dept, Semester, DOB, Address -->
    </div>
    <div class="view" id="view-timetable">
      <!-- Mon–Fri × Period grid -->
      <!-- Each slot colored by attendance: Present/Absent/No-Record/Upcoming/Lab -->
      <!-- Shows: Subject, Teacher name, Room number, Status icon -->
    </div>
    <div class="view" id="view-subjects">
      <!-- Table: Code | Name | Credits | Type | Teacher | Email -->
    </div>


    <div class="view" id="view-teachers">
```

```
  <!-- Card grid: Teacher avatar, Name, Designation, Subjects, Email -->
</div>
<div class="view" id="view-attendance">
  <!-- Subject-wise cards with circular % + progress bar (Green≥85/Amber≥75/Red) -->
  <!-- Detailed log table: Date | Subject | Status -->
</div>
<div class="view" id="view-scan">
  <!-- Face Registration: webcam → capture 5 frames → POST to API -->
  <!-- Shows registered/not-registered status banner -->
</div>
<div class="view" id="view-marks">
  <!-- Table: Subject | CA1–CA5(/10) | Midterm(/50) | Endterm(/100) | Total | Grade -->
  <!-- Grade scale: O≥90% · A+≥80% · A≥70% · B+≥60% · B≥50% · C≥40% · F<40% -->
</div>
<div class="view" id="view-exams">
  <!-- Card grid per exam: Active(amber) / Submitted(green) / Upcoming(blue) / Expired(red) -->
  <!-- Active exams: "Start Exam" button + live countdown timer -->
  <!-- Submitted: shows score obtained -->
</div>
<div class="view" id="view-materials">
  <!-- Card grid: Title | Subject | Teacher | File Type | Open link -->
</div>
<div class="view" id="view-online">
  <!-- Table: Topic | Subject | Teacher | Date | Time | Join link -->
</div>
<div class="view" id="view-notifications">
  <!-- List: unread (blue border) vs read; Mark all read button -->
```

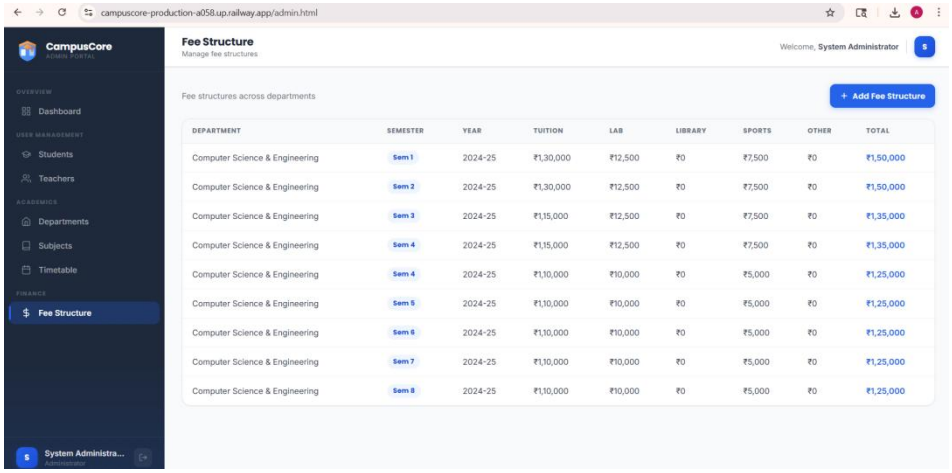
```
    </div>
    <div class="view" id="view-fees">
      <!-- Fee breakdown table + Paid/Balance cards + Pay Now button -->
      <!-- Payment history table: Date | Amount | Mode | Transaction ID -->
    </div>
  </div>
</main>
<!-- MODALS -->
<!-- examMdl  : Question list (MCQ radio / TF radio / Short textarea) + countdown timer -->
<!-- payMdl   : Method selector (UPI/NetBanking/DebitCard/CreditCard) + amount input -->
<script>
const API   = localStorage.getItem('api_base') || window.location.origin;
let token   = '', faceRegStream = null, feePendingAmt = 0;
// ── Init: verify auth, load profile, load dashboard
async function init() {
  token = localStorage.getItem('accessToken') || localStorage.getItem('token') || '';
  if (!token) { location.href = 'auth.html'; return; }
  loadDash();
  const pr = await api('/api/student/profile');
  if (pr.success) document.getElementById('sbName').textContent = pr.profile.FullName;
  checkAndPromptFaceRegistration();  // show banner if face not registered
}
// ── API helper: attaches Bearer token
async function api(ep, opts = {}) {
  const r = await fetch(API + ep, {
    headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + token },
    ...opts
```

```
  });
  if (r.status === 401) { localStorage.clear(); location.href = 'auth.html'; }
  return await r.json().catch(() => ({}));
}
// ── View switcher
function go(view) {
  document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
  document.getElementById('view-' + view).classList.add('active');
  if (view !== 'scan') stopFaceRegistration();
  loadView(view);
}
function loadView(v) {
  const map = {
    dashboard: loadDash,     profile:      loadProfile,
    timetable: loadTT,       subjects:     loadSubjs,
    teachers:  loadTeachers, attendance:   loadAtt,
    marks:     loadMarks,    exams:        loadExams,
    materials: loadMats,     online:       loadOC,
    fees:      loadFees,     notifications: loadNotifications
  };
  if (map[v]) map[v]();
}
// ── Dashboard: stats + today's schedule + attendance summary
async function loadDash() {
  const d = await api('/api/student/dashboard');
  // fill stat cards from d.stats
  // d.stats.todaysSchedule → today's class list
```

```
  // d.stats.subjectAttendance → progress bars
}
// ── Timetable: color-coded grid with attendance overlay
async function loadTT() {
  const [td, ad] = await Promise.all([
    api('/api/student/timetable'),
    api('/api/student/attendance')
  ]);
  // Build attMap: { SubjectID_Date → 'Present'|'Absent' }
  // For each slot determine status: present / absent / no-att / upcoming / lab
  // Render Mon–Fri × Period grid with colored .tt-slot divs
}
// ── Face Registration
async function startFaceRegistration() {
  faceRegStream = await navigator.mediaDevices.getUserMedia({ video: true });
  // Show webcam stream, countdown 3s, then capture 5 frames every 600ms
  // POST /api/student/register-face  body: { images: [base64...] }
}
function stopFaceRegistration() {
  if (faceRegStream) { faceRegStream.getTracks().forEach(t => t.stop()); faceRegStream = null; }
}
async function checkAndPromptFaceRegistration() {
  const d = await api('/api/student/face-status');
  // If d.hasFaceData is false → show warning banner on dashboard
  // If d.needsReRegister is true → prompt to re-register
}
```

```
// ── Marks: table with CA1–CA5, Midterm, Endterm + grade calculation
async function loadMarks() {
  // GET /api/student/marks → d.marks[]
  // Grade: O≥90% A+≥80% A≥70% B+≥60% B≥50% C≥40% F<40%
}
// ── Exams: card grid + live countdown + exam modal
async function loadExams() {
  // GET /api/student/exams → d.exams[]
  // examWindowState(exam) → 'active' | 'submitted' | 'upcoming' | 'expired'
  // Active exams show "Start Exam" button + live timer
}
async function startExam(examId) {
  // GET /api/student/exams/:id → { exam, questions, submissionId }
  // Render questions in modal (MCQ=radio, TF=radio, Short=textarea)
  // Start countdown timer; auto-submit on expire
}
async function submitExam(examId, subId, questionIds) {
  // Collect answers from DOM inputs
  // POST /api/student/exams/:id/submit  body: { answers: [{questionId, answer}] }
  // Toast result: score obtained
}
// ── Fee payment
async function loadFees() {
  // GET /api/student/fees → { feeStructure, totalPaid, pendingAmount, payments[] }
}
async function submitPayment() {
  // POST /api/student/fees/pay
```

```
    // body: { amount, paymentMode: 'UPI'|'NetBanking'|'DebitCard'|'CreditCard', ...extraData }
    // Show success receipt with transactionId
  }
  // ── Notifications
  async function markNotifRead(id) {
    // PUT /api/student/notifications/:id/mark-read
  }
  async function markAllRead() {
    // PUT /api/student/notifications/read-all
  }
</script>
</body>
</html>
```



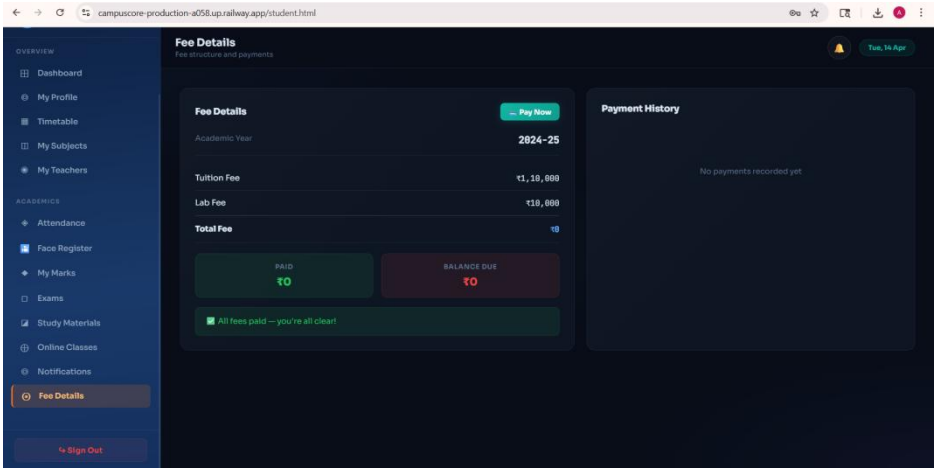


Fig A.7: Fee Management page showing fee structure breakdown and payment history

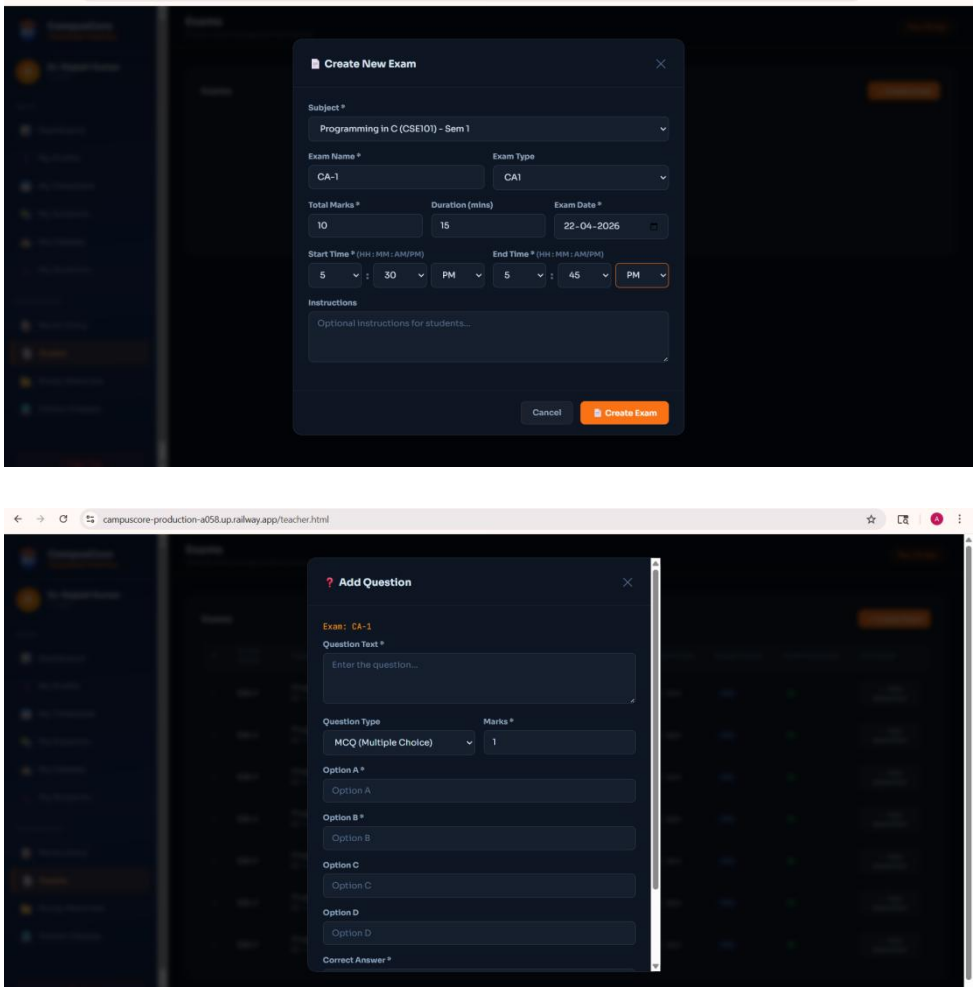


Fig A.8: Online Exam creation form in teacher dashboard

FILE: app.py — CampusCore Flask Backend

```
import os

from datetime import timedelta, datetime

from flask import Flask, jsonify

from flask_jwt_extended import JWTManager

from flask_cors import CORS

from dotenv import load_dotenv

from database import db

load_dotenv()

app = Flask(__name__)

# ── Configuration ──────────────────────────────────────

app.config['SECRET_KEY']              = os.getenv('SECRET_KEY', 'campuscore-secret')

app.config['JWT_SECRET_KEY']          = os.getenv('JWT_SECRET_KEY', 'campuscore-jwt')

app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(hours=24)

app.config['MAX_CONTENT_LENGTH']      = 50 * 1024 * 1024  # 50 MB file upload limit

# ── JWT Error Handlers ────────────────────────────────

jwt = JWTManager(app)

@jwt.expired_token_loader

def expired(h, p):   return jsonify({'error': 'Token expired'}), 401

@jwt.invalid_token_loader

def invalid(e):      return jsonify({'error': 'Invalid token'}), 422

@jwt.unauthorized_loader

def unauth(e):       return jsonify({'error': 'Missing auth header'}), 401

# ── CORS: allow all origins ──────────────────────────

CORS(app, resources={r'/*': {
```

```
    'origins':      '*',
    'methods':      ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    'allow_headers': ['Content-Type', 'Authorization'],
}})
# ── Handle OPTIONS preflight ──────────────────────
@app.before_request
def preflight():
    from flask import request, make_response
    if request.method == 'OPTIONS':
        r = make_response('', 204)
        r.headers['Access-Control-Allow-Origin']  = '*'
        r.headers['Access-Control-Allow-Headers'] = 'Content-Type, Authorization'
        r.headers['Access-Control-Allow-Methods'] = 'GET, POST, PUT, DELETE, OPTIONS'
        return r
# ── Error Handlers ────────────────────────────────────
@app.errorhandler(404)
def e404(e): return jsonify({'error': 'Not found'}), 404
@app.errorhandler(500)
def e500(e): return jsonify({'error': 'Internal server error'}), 500
@app.errorhandler(413)
def e413(e): return jsonify({'error': 'File too large (max 50 MB)'}), 413
# ── Register Blueprints ───────────────────────────
# Each blueprint maps to a route group and a merged routes file
from auth_routes_merged    import bp_auth, bp_common  # /api/auth/* + /api/common/*
from admin_routes_merged   import bp_admin            # /api/admin/*
```

```
from student_routes_merged import bp_student          # /api/student/*

from teacher_routes_merged import bp_teacher, bp_timetable  # /api/teacher/* + /api/timetable/*

app.register_blueprint(bp_auth)

app.register_blueprint(bp_common)

app.register_blueprint(bp_admin)

app.register_blueprint(bp_student)

app.register_blueprint(bp_teacher)

app.register_blueprint(bp_timetable)

# ── Static File Routes ──────────────────────────────

# Flask serves HTML pages directly (no separate web server needed)

@app.route('/')

@app.route('/auth.html')

def index():

    from flask import send_from_directory

    return send_from_directory('.', 'auth.html')

@app.route('/admin.html')

def admin_page():

    from flask import send_from_directory

    return send_from_directory('.', 'admin.html')

@app.route('/student.html')

def student_page():

    from flask import send_from_directory

    return send_from_directory('.', 'student.html')

@app.route('/teacher.html')

def teacher_page():
```

```
    from flask import send_from_directory
    return send_from_directory('.', 'teacher.html')
@app.route('/change_password.html')
def change_password_page():
    from flask import send_from_directory
    return send_from_directory('.', 'change_password.html')
# ── File Upload Serving ──────────────────────────
@app.route('/uploads/<path:filename>')
def serve_upload(filename):
    from flask import send_from_directory
    return send_from_directory('uploads', filename)
# ── Health Check ────────────────────────────────────
@app.route('/health')
def health():
    try:
        db.execute_query('SELECT 1', fetch_one=True)
        db_status = 'connected'
    except Exception as ex:
        db_status = 'error'
    return jsonify({
        'status':   'healthy',
        'version':  '2.0.0',
        'database': db_status,
        'time':     datetime.now().isoformat(),
    })
```

```
# ── Dev Utilities ──────────────────────────────────────

@app.route('/api/test-auth')

def test_auth():

    return jsonify({'success': True, 'message': 'API is working'})

@app.route('/api/routes')

def list_routes():

    # Lists all registered URL rules — useful during development

    routes = [

        {'url': r.rule, 'methods': sorted(r.methods - {'HEAD', 'OPTIONS'})}

        for r in sorted(app.url_map.iter_rules(), key=lambda r: r.rule)

    ]

    return jsonify({'count': len(routes), 'routes': routes})

# ── Entry Point ────────────────────────────────────────

if __name__ == '__main__':

    port  = int(os.getenv('PORT', 5000))

    debug = os.getenv('FLASK_DEBUG', 'False').lower() in ('true', '1')

    app.run(host='0.0.0.0', port=port, debug=debug)

# ── Default Login Credentials (first login = Date of Birth as DD-MM-YYYY) ──

#   Admin:   ADMIN001  →  15-05-1980

#   Teacher: TID001    →  12-03-1975

#   Student: D1S1N01   →  15-08-2006
```

Appendix B: Database Schema Summary

Table B.1: Complete Table List with Row Counts (Seed Data)

#	Table Name	Seed Rows	Primary/Foreign Keys
1	Departments	6	DepartmentID (PK)
2	Subjects	18	SubjectID (PK), DepartmentID (FK)
3	Teachers	8	TeacherID (PK), DepartmentID (FK)
4	Users	9	UserID (PK), UserCode (UNIQUE)
5	Students	10	StudentID (PK), RollNumber (UNIQUE)
6	Timetable	11	TimetableID (PK), TeacherID (FK), SubjectID (FK)
7	TeacherSubjects	11 (auto)	ID (PK), TeacherID (FK), SubjectID (FK)
8	StudentEnrollments	50 (auto)	EnrollmentID (PK), StudentID (FK), TimetableID (FK)
9	Attendance	0	AttendanceID (PK), StudentID (FK), SubjectID (FK)
10	StudentFaceData	0 (transactional)	FaceDataID (PK)
11	FeeStructure	20 (auto)	FeeID (PK)
12	FeePayments	0 (transactional)	PaymentID (PK), StudentID (FK)
13	Marks	0 (transactional)	MarkID (PK)
14	Exams	0 (transactional)	ExamID (PK)
15	ExamQuestions	0 (transactional)	QuestionID (PK), ExamID (FK)
16	ExamSubmissions	0 (transactional)	SubmissionID (PK)
17	ExamAnswers	0 (transactional)	AnswerID (PK)

18	StudyMaterials	0 (transactional)	MaterialID (PK)
19	OnlineClasses	0 (transactional)	OnlineClassID (PK)
20	Notifications	0 (transactional)	NotificationID (PK)
21	ActivityLogs	0 (transactional)	LogID (PK)
22	StudentFaceData	0 (transactional)	FaceDataID (PK), StudentID (UNIQUE FK)

Appendix C: GitHub Repository

The complete source code for CampusCore is available at the following GitHub repository:

URL: https://github.com/campuscore-erp/CampusCore

Repository contents:

- root/app.py — Flask application entry point
- root/database.py — Unified database layer with multi-backend support
- root/auth_routes_merged.py — Authentication and common API blueprint
- root/admin_routes_merged.py — Admin module API blueprint
- root/student_routes_merged.py — Student module API blueprint
- root/teacher_routes_merged.py — Teacher and timetable API blueprints
- root/face_recognition_helper.py — Face recognition utilities (partial)
- root/*.html — Frontend dashboard pages
- root/requirements.txt — Python dependencies
- campuscore_mysql.sql — MySQL schema export
- README.md — Setup and deployment instructions